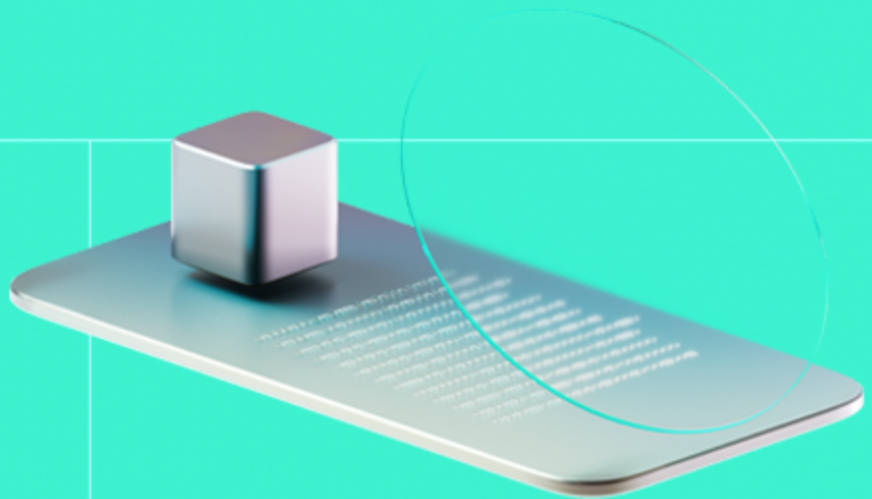




Smart Contract Code Review And Security Analysis Report

Customer: Acecoin

Date: 27/01/2026



We express our gratitude to the Acecoin team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

The Acecoin platform is a subscription-based staking system that allows users to stake ACE tokens and earn monthly rewards through a tiered reward structure.

Document

Name	Smart Contract Code Review and Security Analysis Report for Acecoin
Audited By	Ataberk Yavuzer, Viktor Lavrenenko
Approved By	Panagiotis Konstantinidis
Website	https://acecoin.io/
Changelog	06/01/2026 - Preliminary Report 27/01/2026 - Final Report
Platform	Polygon, Arbitrum, Base, Avalanche
Language	Solidity
Tags	Fungible token, Staking, Incentives
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/AMGAceToken/Ace-SmartContracts
Commit	ac6261b
Remediation Commit	825eaaf

Deployed Contracts

Contract	Network	Address
AceCoin	Polygon	0xcb90576d8dd98ec99db740bd110470a1058ad315
Subscription	Polygon	0xFBD26495461757343E8396CEd6fC98E8043FbC6D
StakingAndRewards	Polygon	0x72496D4550aB06319B9Fbe8638783F07F1840A94

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

36

Total Findings

30

Resolved

6

Accepted

0

Mitigated

Findings by Severity

Severity

Count

Critical

0

High

2

Medium

13

Low

6

Vulnerability	Severity	Status
F-2025-14482 - Staking Capacity Permanently Lost as Stakes Complete	High	Fixed
F-2025-14490 - Team Rewards Accumulate Without Required Active Stake	High	Fixed
F-2025-14449 - Incorrect Assumption of USDT Decimals Leads to Fee Miscalculations	Medium	Fixed
F-2025-14450 - Unstake() Function Behaves Differently Than Documented System Requirement	Medium	Fixed
F-2025-14456 - Testnet Time Constants Used in Production Code	Medium	Fixed
F-2025-14457 - Subscription Renewal Resets Timer Instead of Extending Causing User Fund Loss	Medium	Fixed
F-2025-14458 - Mismatch Between Tier Assignment and Enumeration Breaks Tier Determinism	Medium	Fixed
F-2025-14464 - Custom Fee Change Instantly Invalidates Active Subscriptions	Medium	Accepted
F-2025-14473 - Unregistered Users Can Unstake Without Subscription	Medium	Accepted
F-2025-14475 - Users Can Permanently Block Themselves by Calling unstake() Without Active Stakes	Medium	Accepted
F-2025-14480 - Missing State Validation Enables Re-Subscription to An Active Level	Medium	Fixed
F-2025-14483 - Second Stake Inherits Old Activity Time - Reduced Reward Window	Medium	Accepted
F-2026-14497 - Incorrect Referral Traversal in _addDownlines() Skips Intermediate Uplines	Medium	Fixed

Vulnerability	Severity	Status
F-2026-14501 - Unstake Forfeits Accumulated Rewards Without Documentation/Notice	Medium	Accepted
F-2026-14510 - Unimplemented pendingReward State Variable Results in Dead Code and Potential Reward Accounting Issues	Medium	Fixed
F-2025-14444 - Missing SafeERC20 For Not ERC20 Compliant USDT	Low	Fixed
F-2025-14465 - `getSubscribedLevels()` Returns Expired Levels as Active	Low	Fixed
F-2025-14467 - Missing Minimum Fee Validation in `setDefaultFee()`	Low	Fixed
F-2025-14476 - Missing Maximum Fee Cap in setCustomFee() Allows Arbitrary Fee Assignment	Low	Fixed
F-2026-14502 - Team Rewards Exceeding Daily Limit Are Permanently Deleted	Low	Accepted
F-2026-14506 - Incorrect Boundary Condition Activates Protection System Despite Sufficient Contract Balance	Low	Fixed
F-2025-14427 - Floating Pragma	Info	Fixed
F-2025-14428 - Custom Ownership Implementation Causes Weaker Access Control Guarantees	Info	Fixed
F-2025-14430 - Potential Loss of Ownership Control in Contract Using Ownable	Info	Fixed
F-2025-14431 - Use Of Custom Errors Instead Of Revert Strings	Info	Fixed
F-2025-14434 - Redundant Utility Functions Introduce Dead Code	Info	Fixed
F-2025-14435 - Unused State Variables Introduce Security Risks	Info	Fixed
F-2025-14455 - Stale userStakeTierCount After Unstake Breaks Tier Accounting Invariants	Info	Fixed
F-2025-14466 - `getUserCount()` Returns Misleading Value	Info	Fixed
F-2025-14486 - Lack of Public Access to Critical Protocol Data	Info	Fixed
F-2025-14488 - Duplicate lastStakeAmount Assignment in stake()	Info	Fixed
F-2026-14503 - Precision Loss in Penalty Calculation Allows Penalty-Free Stake And Unstake for Small Amounts	Info	Fixed
F-2026-14504 - Public Functions Not Used Internally Can Lead To Additional Gas Costs	Info	Fixed
F-2026-14508 - Missing Zero-Value Validation for directReward Can Trigger Redundant Transfers	Info	Fixed
F-2026-14509 - Confusing Error Handling When User Has No Active Stakes	Info	Fixed
F-2026-14511 - Redundant Zero-Validation of Pre-Initialized lastClaimTime Wastes Gas Across Reward Functions	Info	Fixed

Documentation quality

- Functional requirements are provided:
 - The project's purpose is described.
 - Business logic is complete.
 - Use cases are provided.
- Technical description is partially provided:
 - Key function descriptions are provided for the files in the scope.
 - Architectural overview is missing.
 - Roles and authorization are described.
 - Information on used technologies is provided.

Code quality

- Best practices are followed.
- The development environment is configured.

Test coverage

Code coverage of the project is **60.5%** (branch coverage):

- Deployment and basic user interactions are covered with tests.
- Negative cases coverage is partially provided.
- Interactions by several users are not tested thoroughly.

Table of Contents

System Overview	7
Privileged Roles	7
Potential Risks	8
Findings	9
Vulnerability Details	9
Disclaimers	108
Appendix 1. Definitions	109
Severities	109
Potential Risks	109
Appendix 2. Scope	110
Appendix 3. Additional Valuables	111

System Overview

AMG AceToken - a subscription-based staking platform with the following contracts:

AceCoin - simple ERC-20 token that mints all initial supply to a deployer. Additional minting is not allowed.

It has the following attributes:

- Name: AceCoin
- Symbol: ACE
- Decimals: 18
- Total supply: 30m tokens.

MemoryManager - a utility smart-contract which provides internal helper functions to group numeric values into fixed-size “tiers” (buckets of 1000), manage them, and paginate over them.

Subscription - the main subscription contract for **AceCoin** ecosystem. Manages user registrations, main subscriptions, level subscriptions (0–11), referral trees up to 12 levels, and fees in an **AceCoin** ecosystem, using USDT for payments. It leverages **MemoryManager** to store downlines in tiered arrays for gas-efficient pagination and counting.

StakingAndRewards - a smart-contract which allows eligible users to stake ACE tokens to earn time-based rewards. Note that during the unstake operation, the user is permanently banned from the system and loses all future access to its functionality.

Privileged roles

- The owner of the **StakingAndRewards** contract can:
 - transfer ownership to another address via **transferOwnership()** function.
 - renounce the ownership via **renounceOwnership()** function.
 - set the new address of the **aceFoundationWallet** via **setAceFoundationWallet()** function.
- The owner of the **Subscription** contract can:
 - set default fee for all users and levels via **setDefaultFee()** function. The default fee can be updated by the owner within the range of 1 to 3 USDT.
 - set a custom fee for a specific user via **setCustomFee()** function. The custom fee can be set within the range of 1 to 100 USDT.
 - update the ace foundation wallet address via **setAceFoundationWallet()** function.
 - transfer ownership to another address via **transferOwnership()** function.
 - renounce the ownership via **renounceOwnership()** function.

Potential Risks

- **Centralized Minting to a Single Address:** The token **AceCoin** contract concentrates minting tokens in a single address, raising the risk of fund mismanagement or theft, especially if key storage security is compromised.
 - However, as of 14/04/2026, it is noted that the total token supply, which was previously held under a single wallet, has been transferred to the Staking & Rewards contract. While this reduces direct exposure associated with a single externally controlled address, the underlying centralization of minting authority remains and should be carefully managed.
- **Lack of Upgradeability:** The project's smart contracts are non-upgradable. While this approach enhances immutability, it also limits the ability to patch vulnerabilities, fix bugs, or introduce improvements after deployment. In the event of unintended behavior or security issues, the absence of an upgrade mechanism significantly reduces the available response options, potentially leading to prolonged downtime or the need for a full contract migration.
- **Administrative Key Control Risks:** The digital contract architecture relies on administrative keys for critical operations. Centralized control over these keys presents a significant security risk, as compromise or misuse can lead to unauthorized actions or loss of funds. It is recommended to use a multi-signature wallet.
 - **Note:** As of 15/04/2026, the deployed contracts utilize a multi-signature wallet for administrative control, which mitigates the risks associated with single-key ownership.

Findings

Vulnerability Details

[F-2025-14482](#) - Staking Capacity Permanently Lost as Stakes Complete - High

Description:

The `totalStaked` counter is never decremented when stakes complete naturally (by reaching the 2x reward limit). This counter is used in critical checks that determine whether new stakes are allowed, meaning over time the system will incorrectly reject valid staking attempts.

Example Scenario:

- 1000 users stake 1000 ACE each = `totalStaked = 1,000,000 ACE`
- All stakes complete (2x claimed), actual locked tokens = 0
- `totalStaked` still shows 1,000,000 ACE
- New user tries to stake 100 ACE but fails because `1,000,100 > contractBalance / 2`

When a stake completes in `_processFIFOStakeInactivity`, only `totalActiveStaked` is decremented:

StakingAndRewards.sol:

```
// Check if stake0 is now complete
if (stake0.totalClaimed >= stake0MaxReward) {
    stake0.active = false;
    _updateCountersForInactiveStake(user, stake0.amount);
    totalActiveStaked -= stake0.amount;
    // BUG: totalStaked is NOT decremented here!
}
```

But `totalStaked` is used in the staking validation:

StakingAndRewards.sol:

```
require(totalStaked + amount <= contractBalance, "Insufficient contract balance");
/* Check for existing active stakes and validate amount */
bool hasActiveStake = activeStakes[msg.sender][0].active ||
    activeStakes[msg.sender][1].active;
if (hasActiveStake) {
    require(amount >= lastStakeAmount[msg.sender], "New stake must be greater than or equal to last stake");
}
```

```

}

uint256 potentialTotalStaked = totalStaked + amount;
require(potentialTotalStaked <= contractBalance / 2, "Total staked would exceed 2x limit");

```

The variable `totalActiveStaked` (which is correctly updated) is never used in these checks.

Impact

1. **Progressive System Degradation:** As more stakes complete naturally, `totalStaked` continues to accumulate while actual locked tokens decrease
2. **Staking Denial of Service:** Eventually, the checks `totalStaked + amount <= contractBalance` and `potentialTotalStaked <= contractBalance / 2` will fail even when the contract has plenty of capacity
3. **Permanent Lockout:** Once `totalStaked` exceeds `contractBalance / 2`, no new stakes can be created, even though the contract may have millions of tokens available
4. **Protocol Death:** The staking system will become completely unusable for new participants once enough stakes complete

Assets:

- StakingAndRewards.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	4/5
Exploitability:	Independent
Complexity:	Simple
Severity:	High

Recommendations

Remediation: Replace `totalStaked` with `totalActiveStaked` in the staking validation checks:

```

function stake(uint256 amount) external onlyActive nonReentrant {
    require(amount > 0, "Cannot stake 0");

    uint256 contractBalance = IERC20(aceToken).balanceOf(address(this));

```

```

    bool protectionSystemActive = isProtectionSystemActive();
    if (protectionSystemActive) {
        revert("Protection system active: New stakes blocked to protect 2x rewards");
    }

    // FIX: Use totalActiveStaked instead of totalStaked
    require(totalActiveStaked + amount <= contractBalance, "Insufficient contract balance");

    bool hasActiveStake = activeStakes[msg.sender][0].active ||
        activeStakes[msg.sender][1].active;
    if (hasActiveStake) {
        require(amount >= lastStakeAmount[msg.sender], "New stake must be greater than or equal to last stake");
    }
    // FIX: Use totalActiveStaked instead of totalStaked
    uint256 potentialTotalActiveStaked = totalActiveStaked + amount;
    require(potentialTotalActiveStaked <= contractBalance / 2, "Total staked would exceed 2x limit");

    // ... rest of function
}

```

Resolution:

Fixed in commit `f9604f9`. The codebase updated `stake()` admission checks to use `totalActiveStaked` (live exposure) rather than `totalStaked`, and re-scoped `totalStaked` to a statistics-only metric, ensuring completed stakes no longer inflate capacity/2x-limit gating and preventing the progressive staking lockout condition over time.

Evidences

PoC

Reproduce:

Run the following command with the PoC test file:

```
forge test --match-test test_ExactFailureAmount -vv
```

Results:

Output:

```
[PASS] test_ExactFailureAmount() (gas: 51816871)
```

Logs:

```
Contract balance: 29380000 ACE
```

totalStaked (ghost counter): 50000 ACE

Maximum stakeable with bug: 14640000 ACE

Maximum stakeable (correct): 14690000 ACE

Difference (lost capacity): 50000 ACE

Trying to stake 14640001 ACE (should fail due to bug):

REVERTED as expected - bug confirmed!

Note: Contract has 29380000 ACE

Could support 2x on: 14690000 ACE of stakes

But bug limits it to: 14640000 ACE

Files:

TotalStakedValidation.t.sol

[F-2025-14490](#) - Team Rewards Accumulate Without Required Active Stake - High

Description:

The documentation states that "**The user must have an active stake**" to receive team rewards. However, `_processTeamRewards()` does NOT check if the referrer has an active stake before accumulating team rewards. That is a gap in the system invariants and the reward-accumulating process. This allows users to:

1. Register and subscribe without staking
2. Accumulate team rewards from downlines' claims
3. Stake minimally later
4. Claim all accumulated team rewards immediately

Impact

- Users can accumulate team rewards without meeting the stake requirement
- They can then stake a minimal amount and claim all accumulated rewards
- This violates the documented requirement and undermines staking incentives

Example Attack:

1. Attacker registers with subscriptions but does not stake anything
2. Builds a referral network of active stakers
3. Accumulates 1000 ACE in team rewards over months (without staking)
4. Stakes 100 ACE (minimum)
5. Claims all 1000 ACE team rewards immediately
6. Effectively gets 10x return without real staking commitment

To receive team rewards:

--The user must have an active stake.

StakingAndRewards.sol:

```
function _processTeamRewards(address user, uint256 claimedAmount) internal {
    . . .
    /* Skip team rewards if referrer is inactive */
    if (!isReferrerActive) continue;

    bool canReceiveReward = false;
    if (level == 0) {
        canReceiveReward = affiliate.isMainActive(referrerId) && affiliate
.isLevelActive(referrerId, 0);
    } else {
```

```

        canReceiveReward = affiliate.isMainActive(referrerId) && affiliate
        .isLevelActive(referrerId, level);
    }

    if (!canReceiveReward) continue;

    // ! MISSING CHECK: Does referrer have an active stake?
    // Documentation says: "The user must have an active stake" to receive
    team rewards

    uint256 teamReward = (claimedAmount * TEAM_REWARD_PERCENT) / PERCENT_D
    ENOMINATOR;

    TeamReward storage reward = teamRewards[referrerAddress][level];
    reward.totalReward += teamReward;
    // ...
}
}

```

There is **no verification** that **the referrer has an active stake before accumulating team rewards**.

Assets:

- StakingAndRewards.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	4/5
Exploitability:	Independent
Complexity:	Simple
Severity:	High

Recommendations

Remediation:

Add a check for an active stake in `_processTeamRewards()`:

```

function _processTeamRewards(address user, uint256 claimedAmount) internal {
    if (isProtectionSystemActive()) {
        return;
    }
}

```

```

    }

    uint256 userId = affiliate.userIds(user);
    uint256[12] memory referrers = affiliate.getReferrers(userId);
    uint256 currentTime = block.timestamp;

    for (uint256 level = 0; level < 12; level++) {
        uint256 referrerId = referrers[level];
        if (referrerId == 0) continue;

        address referrerAddress = affiliate.getUserAddress(referrerId);
        if (referrerAddress == address(0)) continue;
        if (hasUnstaked[referrerAddress]) continue;

        // FIX: Check if referrer has an active stake
        bool hasActiveStake = activeStakes[referrerAddress][0].active ||
                               activeStakes[referrerAddress][1].active;
        if (!hasActiveStake) continue; // Skip if no active stake

        // ... rest of checks and processing
    }
}

```

Resolution:

Fixed in commit [f9604f9](#). The codebase added an explicit active-stake requirement in `_processTeamRewards()` by checking whether the referrer has at least one active stake (slot 0 or slot 1) before accumulating team rewards, aligning reward accrual with the documented “active stake required” requirement.

Evidences

PoC

Reproduce:

Run the following command with the PoC file:

```
forge test --match-test test_StakeLaterAndClaimAccumulatedRewards -vv
```

Results:

Output:

```
[PASS] test_StakeLaterAndClaimAccumulatedRewards() (gas: 2968136)
```

Logs:

```
=== Stake Later and Claim Accumulated Rewards ===
```

```
Step 1: Referrer registered (NO stake)
```

Step 2: Downline claimed rewards 5 times

Referrer accumulated: 16 ACE in team rewards
(without having staked!)

Step 3: Referrer stakes 100 ACE

Has active stake: true

Step 4: Referrer claims rewards

Received: 16 ACE

[F-2025-14449](#) - Incorrect Assumption of USDT Decimals Leads to Fee Miscalculations - Medium

Description:

The protocol makes a **fundamental and incorrect assumption** about the decimal precision of the USDT token by treating it as an 18-decimal ERC20 asset. In reality, USDT uses **6 decimals**, and this discrepancy introduces severe inconsistencies in all fee-related logic across the system.

This incorrect assumption is embedded directly into the contract design through hardcoded constants and fee calculations that rely on 1e18 precision. For example, the `Subscription` contract defines fee limits and defaults using 18-decimal units:

```
uint256 public constant MAX_FEE = 3 * 1e18; /* 3 USDT (18 decimals) */  
...  
defaultFee = 1e18;
```

This implementation implicitly assumes that 1 USDT == 1e18, which is false. On-chain USDT represents:

```
1 USDT = 1e6
```

As a result, all protocol fees are effectively scaled by **12 additional orders of magnitude** beyond their intended value. Any logic that relies on these values - including registration, subscription renewal, level access, and custom fee configuration- operates on incorrect economic assumptions.

Because ERC20 tokens do not enforce a standard decimal size, this issue is **not a theoretical edge case** but a concrete integration flaw that manifests immediately when interacting with a real USDT contract.

This vulnerability leads to **protocol-wide economic breakage**, including:

- Users being required to approve and transfer impossibly large USDT amounts as fees
- Transactions reverting due to insufficient balances or allowances
- Inability to onboard users or renew subscriptions
- Incorrect fee reporting in off-chain services and UIs

Assets:

- `Subscription.sol` [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: It is recommended to:

- replace the incorrect hardcoded number of decimals for USDT:

```
uint256 public constant MAX_FEE = 3 * 1e18;  
...  
defaultFee = 1e18;
```

with

```
uint256 public constant MAX_FEE = 3 * 1e6;  
...  
defaultFee = 1e6;
```

- edit the provided natspec comment to ensure it describes the correct system maximum limits

```
3 USDT (6 decimals)
```

- clearly document decimal handling expectations for integrators and frontends

Resolution: **Fixed** in [f9604f9](#). The incorrect 18 decimals was replaced with the correct number 6 to represent the appropriate decimals number of the USDT token.

[F-2025-14450](#) - Unstake() Function Behaves Differently Than Documented System Requirement - Medium

Description:

The project documentation explicitly claims that the contract verifies its token balance on every stake and claim/withdraw operation.

The `contract balance` is checked on every stake and claim/withdraw operation.

However, this guarantee is not upheld in the implementation of `StakingAndRewards::unstake()` function, which performs no explicit contract-balance validation prior to transferring tokens back to the user. Specifically, the `stake()` function enforces strict balance and safety constraints, including total staking caps and protection-system gating,

```
function stake(uint256 amount) external onlyActive nonReentrant {
    require(amount > 0, "Cannot stake 0");

    /* Cache contract balance for gas efficiency */
    uint256 contractBalance = IERC20(aceToken).balanceOf(address(this));

    /* Check if new stakes are allowed under protection system */
    bool protectionSystemActive = isProtectionSystemActive();
    if (protectionSystemActive) {
        revert("Protection system active: New stakes blocked to protect 2
x rewards");
    }
    require(totalStaked + amount <= contractBalance, "Insufficient contra
ct balance");
    ...
}
```

while `claimRewards()` routes through additional safeguards when liquidity is constrained.

```
function claimRewards() external onlyActive nonReentrant {
    require(block.timestamp >= lastClaimTime[msg.sender] + HOURS, "Cooldo
wn active");

    /* Cache user data to avoid multiple storage reads */
    address user = msg.sender;
    uint256 userTotalStaked = 0;
    uint256 totalClaimed = 0;
```

```

/* Calculate user totals in gas-efficient loop */
for (uint i = 0; i < 2; i++) {
    if (activeStakes[user][i].active) {
        userTotalStaked += activeStakes[user][i].amount;
        totalClaimed += activeStakes[user][i].totalClaimed;
    }
}

require(totalClaimed < (userTotalStaked * MAX_REWARD_MULTIPLIER) / PERCENT_DENOMINATOR, "Maximum rewards claimed");

uint256 stakingReward = _calculateAllRewards(user);

/* Check if team rewards and foundation unlock are allowed under protection system */
bool protectionSystemActive = isProtectionSystemActive();
uint256 teamReward = 0;
uint256 foundationUnlock = 0;

if (stakingReward > 0 && !protectionSystemActive) {
    _processTeamRewards(user, stakingReward);
}

if (!protectionSystemActive) {
    teamReward = _calculateTeamRewards(user);
    foundationUnlock = ((stakingReward + teamReward) * FOUNDATION_UNLOCK) / PERCENT_DENOMINATOR;
}
...
}

```

In contrast, `unstake()` directly attempts to transfer the user's principal without first ensuring that the contract holds sufficient ACE balance.

Such discrepancies between the function's intended behavior and its documentation can cause confusion for integrators, auditors, or contributors who rely on NatSpec or other documentation for understanding the external interface of the protocol.

Assets:

- StakingAndRewards.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate:

2/5

Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: It is recommended to fix the mismatch between the documentation and implementation by either editing the documentation or introducing the missing protection features.

Resolution: **Fixed** in commit `f9604f9`. `unstake()` now explicitly validates the contract's `aceToken` balance before transferring `returnAmount` to the user, aligning the implementation with the documented guarantee that balance checks occur on stake and claim/withdraw flows and preventing misleading assumptions by integrators and reviewers.

F-2025-14456 - Testnet Time Constants Used in Production Code - Medium

Description:

The contracts `Subscription.sol` and `StakingAndRewards.sol` use testnet time duration constants instead of mainnet values. All time-related constants are configured for testing purposes (minutes/seconds) rather than production deployment (days/hours), causing the entire economic model and subscription system to malfunction on mainnet deployment.

This vulnerability has severe economic and operational consequences:

Subscription System:

- Users pay for a "monthly" subscription that expires in **30 minutes** instead of **30 days**
- Users must renew subscriptions **1,440 times per month** (every 30 minutes)
- Completely unusable subscription model on mainnet

Rewards System:

- Monthly rewards (10%/month) are calculated over **15 minutes** instead of **30 days**
- Rewards accrue **faster** than intended
- Contract funds would be drained almost immediately

Subscription.sol:

```
uint256 public constant MAX_FEE = 3 * 1e6; /* 3 USDT (6 decimals) */
uint256 public constant SUBSCRIPTION_PERIOD = 30 minutes; /* 30 days */ // ←
BUG: Should be 30 days
```

StakingAndRewards.sol:

```
uint256 public constant MAX_DAILY_TEAM_REWARD = 30e18;
uint256 public constant MAX_REWARD_MULTIPLIER = 20000;
uint256 private constant REWARD_DURATION = 30 minutes; // ← BUG: Should be 30
days
uint256 private constant DAY = 30 seconds; // Should be 1 days
```

Assets:

- StakingAndRewards.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]
- Subscription.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Consider identifying and updating all testnet time constants to mainnet values as intended. For instance:

```
uint256 private constant REWARD_DURATION = 30 days;
```

```
uint256 public constant SUBSCRIPTION_PERIOD = 30 days;  
uint256 private constant DAY = 1 days;
```

Resolution: Fixed in commit [f9604f9](#). The codebase replaced the testnet-only time constants with production durations, setting `SUBSCRIPTION_PERIOD` and `REWARD_DURATION` to `30 days` and defining `HOURS/DAY/MONTH/MAX_INACTIVE_PERIOD` using standard hour/day-based values to restore the intended subscription and rewards economics.

[F-2025-14457](#) - Subscription Renewal Resets Timer Instead of Extending Causing User Fund Loss - Medium

Description:

The `renewSubscription()` and `subscribeToLevel()` functions reset subscription timers to the current timestamp instead of extending from the existing expiry time. This causes users who renew early (before expiration) to lose their remaining paid subscription time and waste their payment.

Affected Functions:

- `Subscription.sol::renewSubscription()`
- `Subscription.sol::subscribeToLevel()`

Users lose money and subscription time when renewing before expiration:

Early Main Subscription Renewal

- Day 0: User pays 1 USDT, subscription valid until Day 30
- Day 5: User renews (pays another 1 USDT)
 - **Expected:** Subscription extends to Day 60 (30 + 30)
 - **Actual:** Subscription resets to Day 35 (5 + 30)
 - **Loss:** 25 days (worth 0.83 USDT wasted)

Subscription.sol (renew):

```
function renewSubscription() external nonReentrant {
    uint256 userId = userIds[msg.sender];
    require(userId != 0, "Not registered");
    User storage user = users[userId];
    uint256 fee = getRegistrationFee(msg.sender);

    if (fee > 0 && aceFoundationWallet != address(0)) {
        require(usdtToken.balanceOf(msg.sender) >= fee, "Insufficient USDT balance");
        require(usdtToken.allowance(msg.sender, address(this)) >= fee, "Insufficient USDT allowance");
        require(usdtToken.transferFrom(msg.sender, aceFoundationWallet, fee), "ERC20: transfer failed");
    }

    user.lastMainPayment = block.timestamp; // ! Always resets to now, doesn't extend

    _updateActiveStatus(userId);
    emit SubscriptionRenewed(userId, fee);
}
```

Subscription.sol (subscribe to specific level):

```
function subscribeToLevel(uint256 level) external validLevel(level) nonReentrant {
    uint256 userId = userIds[msg.sender];
    require(userId != 0, "Not registered");
    User storage user = users[userId];
    require(isMainActive(userId), "Main subscription inactive");
    require(level == 0 || user.levelInfo[level-1].isActive, "Previous level inactive");

    uint256 fee = getLevelFee(msg.sender, level);
    if (fee > 0 && aceFoundationWallet != address(0)) {
        require(usdtToken.balanceOf(msg.sender) >= fee, "Insufficient USDT balance");
        require(usdtToken.allowance(msg.sender, address(this)) >= fee, "Insufficient USDT allowance");
        require(usdtToken.transferFrom(msg.sender, aceFoundationWallet, fee), "ERC20: transfer failed");
    }

    user.levelInfo[level] = LevelInfo(block.timestamp, true); // ! Resets timer
    emit LevelSubscribed(userId, level, fee);
}
```

Assets:

- Subscription.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	4/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation:

Consider extending the subscription when someone makes a payment, instead of resetting the subscription timer.

Resolution:

Fixed in `c9d413a`. Both `renewSubscription()` and `subscribeToLevel()` have been protected and now can only be executed when the main or level subscription is over:

`renewSubscription()`

```
function renewSubscription() external nonReentrant {  
    ...  
    if (isMainActive(userId)) revert SubscriptionAlreadyActive();  
    ...  
}
```

`subscribeToLevel()`

```
function subscribeToLevel(uint256 level) external validLevel(level) nonReentrant {  
    ...  
    if (isLevelActive(userId, level)) revert LevelAlreadySubscribed();  
    ...  
}
```

[F-2025-14458](#) - Mismatch Between Tier Assignment and Enumeration Breaks Tier Determinism - Medium

Description:

The `MemoryManager` contract uses **two fundamentally different approaches** to handle tiering, which causes **critical inconsistency** in tier calculation and enumeration logic.

- **Value-Based Tiering** during insertion: stake tiers and downline tiers are determined by dividing the value (e.g., `stakeld`) by a constant `TIER_SIZE`

```
function addToTier(mapping(uint256 => uint256[]) storage tiers, uint256 value) internal {
    uint256 tier = value / TIER_SIZE;
    tiers[tier].push(value);
}
```

- **Count-Based Tier Enumeration** during tier counting: the number of tiers is estimated by dividing the count of stakes or downlines by `TIER_SIZE` with rounding:

```
function getTierCount(uint256 totalCount) internal pure returns (uint256) {
    return (totalCount + TIER_SIZE - 1) / TIER_SIZE;
}
```

For Example:

Consider Alice staking twice with `stakeld` values 1999 and 2000:

```
1999 → assigned to tier 1 (since 1999 / 1000 = 1)
2000 → assigned to tier 2 (since 2000 / 1000 = 2)
```

Since:

```
userStakeCount[msg.sender] == 2
getTierCount(2) => (totalCount + TIER_SIZE - 1) / TIER_SIZE => 2 + 1000 - 1 / 1000 = 1
```

This result incorrectly implies only **one tier is used**, which conflicts with the actual tier assignments.

This design mismatch leads to **incorrect and non-deterministic tier counts** that do not reflect:

1. **The actual distribution of stake ids among tiers** in the `StakingAndRewards::stake()` function and leads to the incorrect calculation of the `maxTiers` number that affects the paginated data generation process in the `MemoryManager::getPaginatedData()` function that has a negative impact on the `StakingAndRewards::getUserStakeIds()` and `StakingAndRewards::getUserStakeIdsPaginated()` functions output data.
2. **The actual distribution of downlines among downline tiers** in the `Subscription::_addDownlines()` which is utilized in the `Subscription::register()` function.

Although the issue does not directly impact the financial components of the system, this discrepancy undermines the reliability and integrity of the tiering mechanism and may lead to downstream issues for system integrators, including frontend components and decentralized applications.

Assets:

- MemoryManager.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	4/5
Exploitability:	Independent
Complexity:	Complex
Severity:	Medium

Recommendations

Remediation:

It is recommended to:

- redesign the tier calculation logic to take into account situations when user stakes are in different tiers.
- provide a proper test suite covering all edge cases of the new functionality.
- document the intended behavior of the system.

Resolution:

Fixed in `825eaaf`. The `_addDownlines()` function was redesigned to properly calculate the `downlineTierCount` value as follows:

```
...  
uint256 actualTier = newUserId / TIER_SIZE; // Same formula as addToTier()
```

```

line 30
    if (!tierUsed[current.id][level][actualTier]) {
        tierUsed[current.id][level][actualTier] = true; // Mark tier
as used
        downlineTierCount[current.id][level]++; // Increment unique
tier count
    }
    ...

```

In addition to that the functions `MemoryManager::getTierCount()`, `MemoryManager::getPaginatedData()`, `StakingAndRewards::getUserStakeIds()` and `StakingAndRewards::getUserStakeIdsPaginated()` were removed from the codebase. The calculation of the `userStakeTierCount` was removed from the `StakingAndRewards::stake()` function.

Evidences

PoC

Reproduce:

Run the PoC file with the following command:

```
forge test --match-test test_StakeInvisibilityAfter1000Stakes -vvv
```

Results:

Output:

```

[PASS] test_StakeInvisibilityAfter1000Stakes() (gas: 762917999)
Logs:
=== TIER MISMATCH EXPLOIT PoC ===

Step 1: Creating 1001 users and having them stake...
Current nextStakeId: 1002
Stakes created: 1001

Step 2: Creating victim user who stakes after stakeId > 1000...
Victim's stakeId: 1002
Victim's stakeId tier: 1 (stakeId / 1000)

Step 3: Verifying stake exists via getStakeInfo()...
Direct query - getStakeInfo(victim, 0):
Amount: 100
Active: true

Step 4: Querying via getUserStakeIds() - EXPLOIT DEMO...
getUserStakeIds(victim) returned: 0 stakes

```

!!! VULNERABILITY CONFIRMED !!!

Stake EXISTS and is ACTIVE but getUserStakeIds() returns EMPTY!

User's stake is INVISIBLE to pagination queries!

=== ROOT CAUSE ANALYSIS ===

Victim has 1 stake, so userStakeCount = 1

$\text{getTierCount}(1) = (1 + 999) / 1000 = 1$

Pagination loop: for (tier = 0; tier < 1; tier++)

Only tier 0 is checked!

But stakeId 1002 is in tier 1 (stakeId / 1000)!

Tier 1 is NEVER checked, stake is INVISIBLE!

Files:

TierMismatchExploit.t.sol

[F-2025-14464](#) - Custom Fee Change Instantly Invalidates Active Subscriptions - Medium

Description:

The `isMainActive()` and `isLevelActive()` functions contain flawed logic that instantly invalidates a user's active subscription whenever the admin changes their custom fee. This causes users to lose paid subscription time and blocks access to all staking operations, resulting in direct financial loss.

Vulnerable Functions:

- `Subscription.sol::isMainActive()`
- `Subscription.sol::isLevelActive()`
- `Subscription.sol::setCustomFee()`

Direct Financial Loss

Scenario: Admin changes user's custom fee during an active subscription

1. **Day 0:** User pays 1 USDT → subscription valid until Day 30
2. **Day 10:** Admin changes user's custom fee to 2 USDT
3. **Instant Effect:** User's subscription becomes **INVALID** immediately
4. **Result:** User loses 20 days they already paid for (0.67 USDT wasted)
5. **To Continue:** User must pay 2 USDT again

Total Cost: 3 USDT for 40 days (should be 2 USDT for 60 days)

Staking Operations Blocked

Users with active stakes cannot claim rewards when their custom fee is changed:

```
User has 1000 ACE staked with pending rewards
↓
Admin changes custom fee
↓
Subscription instantly invalid
↓
Cannot claim rewards (reverts with "Inactive subscription")
↓
Funds effectively frozen until user pays new fee
```

Subscription.sol:

```
function isMainActive(uint256 userId) public view returns (bool) {
    if (userId == GENESIS_ID) return true;
    User storage user = users[userId];
```

```
// ! Instantly invalidates subscription
if (hasCustomFee[user.userAddress] &&
    customFeeChangeTimestamp[user.userAddress] > user.lastMainPayment) {
    return false; // ← Returns false even if subscription should be valid for 20 more days!
}

return user.lastMainPayment + SUBSCRIPTION_PERIOD >= block.timestamp;
}
```

If `customFeeChangeTimestamp > lastMainPayment`, subscription is marked as inactive regardless of remaining time.

Also, the `isLevelActive()` function has the same flaw as it was also designed in the same way.

Assets:

- Subscription.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Accepted

Classification

Impact Rate:	4/5
Likelihood Rate:	5/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: Consider keeping existing subscriptions until their expiry. For instance:

```
function isMainActive(uint256 userId) public view returns (bool) {
    if (userId == GENESIS_ID) return true;
    User storage user = users[userId];

    // Check if subscription period is still valid
    uint256 expiryTime = user.lastMainPayment + SUBSCRIPTION_PERIOD;
    bool periodValid = expiryTime >= block.timestamp;

    // REMOVED: Instant invalidation logic
    // NEW: Custom fee changes only affect FUTURE renewals
}
```

```

        return periodValid;
    }

    function isLevelActive(uint256 userId, uint256 level) public view validLevel(
    level) returns (bool) {
        if (userId == GENESIS_ID) return true;
        User storage user = users[userId];

        if (!user.levelInfo[level].isActive) return false;

        // Check if level subscription period is still valid
        uint256 expiryTime = user.levelInfo[level].lastPayment + SUBSCRIPTION_PER
        IOD;
        return expiryTime >= block.timestamp;

        // REMOVED: Instant invalidation logic
    }

```

Resolution:

The finding was **acknowledged** by the client with the following statement:

This is intentional. The purpose of this admin right was to enable penalization in cases where a user violates our terms and policies or engages in misconduct. By increasing the user's fee, the admin can impose a penalty. This change was designed to be immediate, allowing swift enforcement as soon as the admin initiates the action.

If a user does not agree with the imposed penalty, they have the option to unstake and exit the system, as an active subscription is not required during the unstaking process.

Users who unstake will get back their capital after the regular penalties that are imposed on unstake, users can do that without an active subscription.

Now we have set a minimum limit of 1 USDT and a maximum limit of 100 USDT for this admin right.

[F-2025-14473](#) - Unregistered Users Can Unstake Without Subscription - Medium

Description:

The `unstake()` function in `StakingAndRewards.sol` lacks the `onlyActive` modifier, allowing users to unstake their tokens and recover 80% of their stake **without an active subscription**. This violates the documented system requirement that "users cannot withdraw rewards" when their subscription is inactive. While `stake()` and `claimRewards()` correctly enforce subscription checks, the inconsistent protection on `unstake()` creates a critical system invariant violation and allows users to bypass subscription requirements when exiting their positions.

Documentation Requirement:

If a user's subscription is inactive, they cannot stake and cannot claim or withdraw rewards

The `unstake()` function **withdraws the principal stake amount** (minus penalties), which qualifies as a "withdrawal" operation. Therefore, it should require an active subscription like `stake()` and `claimRewards()` do. This seems like an intended behavior. However, as a best practice, the modifier should prevent users from calling the unstake function when they do not have any active stakes.

StakingAndRewards.sol (missing modifier):

```
function unstake() external {
    require(!hasUnstaked[msg.sender], "Already unstaked");

    address user = msg.sender;
    uint256 totalStakeAmount = 0;
    uint256 totalClaimedRewards = 0;

    /* Calculate totals in one pass for gas efficiency */
    Stake storage stake0 = activeStakes[user][0];
    Stake storage stake1 = activeStakes[user][1];

    bool stake0WasActive = stake0.active;
    bool stake1WasActive = stake1.active;

    . . .
}
```

The function has NO `onlyActive` modifier. Both `stake()` and `claimRewards()` functions have this modifier for extra protection.

Assets:

- StakingAndRewards.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]
- Subscription.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Accepted

Classification

Impact Rate:	2/5
Likelihood Rate:	4/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation:

Add the `onlyActive` modifier to the `unstake()` function to enforce subscription validation:

```
function unstake() external onlyActive { // ! Add onlyActive modifier
    require(!hasUnstaked[msg.sender], "Already unstaked");

    address user = msg.sender;
    // ... rest of function unchanged ...
}
```

Resolution:

The finding was **acknowledged** by the client with the following statement:

Design Intent

The current behavior is intentional business logic, not a vulnerability.

We distinguish clearly between:

- Earning / receiving rewards (requires active subscription):
- ``stake()`` → ``onlyActive``
- ``claimRewards()`` → ``onlyActive``
- Exiting the system and withdrawing principal with penalty:
- ``unstake()`` → does not require an active subscription

The rule we enforce is:

- If the subscription is inactive, a user:
- Cannot stake
- Cannot claim or receive rewards
- Can still exit (unstake) and recover penalized principal

So the phrase in the documentation:

"If a user's subscription is inactive, they cannot stake and cannot claim or withdraw rewards"

is intended to apply to rewards, not to principal return on exit.

Why `unstake()` is not gated by `onlyActive`

1. Exit is allowed regardless of subscription status
`unstake()` is designed as a permanent exit:
 - Applies 20% penalty on principal
 - Deducts already claimed rewards
 - Forfeits any unclaimed rewards and team rewards
 - Sets `hasUnstaked[user] = true` (account permanently disabled)

Given this, we want users to always be able to:

- Leave the system
- Recover the remaining portion of their stake
- Even if their subscription has expired

2. No rewards are paid in `unstake()`

`unstake()`:

- Does not calculate or transfer any staking or team rewards
- Only returns penalized principal:

solidity

```
uint256 penalty = (totalStakeAmount * 20) / 100;
```

```
uint256 totalDeductions = penalty + totalClaimedRewards;
```

```
uint256 returnAmount = totalStakeAmount > totalDeductions ?  
totalStakeAmount - totalDeductions : 0;
```

- Therefore, it does not violate the rule about "cannot claim or withdraw rewards" when inactive.

Planned Documentation Update

We agree that the current wording in the documentation can be interpreted too broadly.

Evidences

PoC

Reproduce:

Test Case:

```

function testUnstake_WithExpiredSubscription() public {
    console.log("\n=== UNSTAKE WITH EXPIRED SUBSCRIPTION TEST ===\n");

    vm.startPrank(user);

    console.log("Step 1: Register and stake with active subscription");
    usdtToken.approve(address(subscription), 1e18);
    subscription.register(1);

    aceToken.approve(address(staking), 100e18);
    staking.stake(100e18);

    uint256 userId = subscription.userIds(user);
    console.log(" - User ID:", userId);
    assertTrue(userId != 0, "User should be registered");
    assertTrue(subscription.isMainActive(userId), "User should have active subscription");

    console.log("\nStep 2: Wait for subscription to expire (30 days + 1)"
);
    vm.warp(block.timestamp + 30 days + 1);

    bool isActive = subscription.isMainActive(userId);
    console.log(" - Subscription active:", isActive);
    assertFalse(isActive, "Subscription should be expired");

    console.log("\nStep 3: Try to stake with expired subscription - should fail");
    aceToken.approve(address(staking), 100e18);
    vm.expectRevert("Inactive subscription");
    staking.stake(100e18);
    console.log(" - Stake correctly blocked [OK]");

    console.log("\nStep 4: Try to claim with expired subscription - should fail");
    vm.expectRevert("Inactive subscription");
    staking.claimRewards();
    console.log(" - Claim correctly blocked [OK]");

    console.log("\nStep 5: Try to unstake with expired subscription");

    uint256 balanceBefore = aceToken.balanceOf(user);

    // This should fail if unstake() requires active subscription
    // But it SUCCEEDS because unstake() has NO onlyActive modifier!
    staking.unstake();

    uint256 balanceAfter = aceToken.balanceOf(user);

```

```

uint256 returned = balanceAfter - balanceBefore;

console.log(" - Amount returned:", returned);
assertEq(returned, 80e18, "Should receive 80 ACE");

vm.stopPrank();
}

```

Results:

Output:

=== UNSTAKE WITH EXPIRED SUBSCRIPTION TEST ===

Step 1: Register and stake with active subscription

- User ID: 5000

Step 2: Wait for subscription to expire (30 days + 1)

- Subscription active: false

Step 3: Try to stake with expired subscription - should fail

- Stake correctly blocked [OK]

Step 4: Try to claim with expired subscription - should fail

- Claim correctly blocked [OK]

Step 5: Try to unstake with expired subscription

- Amount returned: 800000000000000000

[F-2025-14475](#) - Users Can Permanently Block Themselves by Calling `unstake()` Without Active Stakes - Medium

Description:

The `unstake()` function in `StakingAndRewards.sol` does not verify that users have active stakes before executing. This allows any user to call `unstake()` even when they have never staked, permanently setting `hasUnstaked[msg.sender] = true` and blocking themselves from ever staking in the future.

While this is primarily a self-griefing vulnerability, it has several concerning implications:

1. **Permanent Account Lockout:** Users who accidentally call `unstake()` without having staked are permanently blocked from participating in the staking system
2. **Irreversible:** There is no admin function to reset the `hasUnstaked` flag, making this damage permanent

The vulnerability violates the principle of fail-safe defaults and allows state changes that provide no value while causing permanent harm.

StakingAndRewards.sol:

```
function unstake() external {
    require(!hasUnstaked[msg.sender], "Already unstaked");

    address user = msg.sender;
    uint256 totalStakeAmount = 0;
    uint256 totalClaimedRewards = 0;

    /* Calculate totals in one pass for gas efficiency */
    // Cache storage reads outside loop for gas efficiency
    Stake storage stake0 = activeStakes[user][0];
    Stake storage stake1 = activeStakes[user][1];

    bool stake0WasActive = stake0.active;
    bool stake1WasActive = stake1.active;

    if (stake0.active) {
        totalStakeAmount += stake0.amount;
        totalClaimedRewards += stake0.totalClaimed;
        stake0.active = false;
    }

    if (stake1.active) {
        totalStakeAmount += stake1.amount;
        totalClaimedRewards += stake1.totalClaimed;
    }
}
```

```

        stake1.active = false;
    }

    /* Remove stakes from tier system */
    if (stake0WasActive) {
        removeFromTier(userStakeTiers[user], stake0.stakeId);
        userStakeCount[user]--;
    }
    if (stake1WasActive) {
        removeFromTier(userStakeTiers[user], stake1.stakeId);
        userStakeCount[user]--;
    }

    /* Use efficient math for penalty and deductions */
    uint256 penalty = (totalStakeAmount * 20) / 100;
    uint256 totalDeductions = penalty + totalClaimedRewards;
    uint256 returnAmount = totalStakeAmount > totalDeductions ? totalStakeAmount - totalDeductions : 0;

    /* Batch state updates for unstake */
    hasUnstaked[user] = true; // ! This is set even when totalStakeAmount = 0
    . . .
}

```

Once `hasUnstaked` is set to `true`, the user can never call `stake()` or `claimRewards()` again.

Assets:

- StakingAndRewards.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Accepted

Classification

Impact Rate:	4/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation:

Add a check to ensure users have at least one active stake before allowing them to unstake:

```
function unstake() external {
    require(!hasUnstaked[msg.sender], "Already unstaked");

    address user = msg.sender;

    // Check if user has any active stakes
    Stake storage stake0 = activeStakes[user][0];
    Stake storage stake1 = activeStakes[user][1];
    require(stake0.active || stake1.active, "No active stakes to unstake");

    uint256 totalStakeAmount = 0;
    uint256 totalClaimedRewards = 0;

    // ... rest of the function
}
```

Resolution:

The finding was **acknowledged** by the client with the following statement:

Design Intent: Permanent Exit Mechanism

The unstake() function is designed as a permanent exit from the staking system. When a user calls unstake(), they are intentionally leaving the system, which is why the account is permanently disabled.

Behavior:

- Sets `hasUnstaked[user] = true` permanently
- Prevents future staking from that account
- This is by design, not a bug

Why This Design:

1. Permanent Exit Intent:

- unstake() is meant for users who want to permanently leave
- The permanent disable reflects this intent
- Users understand they are exiting the system

2. System Integrity:

- Prevents re-entry after exit
- Maintains clear separation between active participants and those who have exited
- Aligns with the protocol's economic model

3. User Choice:

- Users choose to call unstake() knowing it is permanent
- This is a deliberate action, not accidental
- The function name and behavior communicate permanence

Conclusion:

The current behavior is intentional business logic:

- `unstake()` is a permanent exit mechanism
- Permanent account disable is by design
- No recovery mechanism is intentional
- Users understand they are leaving the system permanently

F-2025-14480 - Missing State Validation Enables Re-Subscription to An Active Level - Medium

Description:

The `subscribeToLevel(uint256 level)` function allows a user to subscribe to the same level multiple times without restriction. The function does not verify whether the target level is already active for the caller before processing the subscription and charging the fee.

```
function subscribeToLevel(uint256 level) external validLevel(level) nonReentrant {
    uint256 userId = userIds[msg.sender];
    require(userId != 0, "Not registered");
    User storage user = users[userId];
    require(isMainActive(userId), "Main subscription inactive");
    require(level == 0 || user.levelInfo[level-1].isActive, "Previous level is active");

    uint256 fee = getLevelFee(msg.sender, level);
    ...
    user.levelInfo[level] = LevelInfo(block.timestamp, true);
    emit LevelSubscribed(userId, level, fee);
}
```

There is **no check** ensuring that the level is not active prior to execution. As a result, an already active level can be subscribed to again, overwriting the existing `LevelInfo()` and recharging the fee.

Assets:

- Subscription.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation:

It is recommended to explicitly prevent re-subscription to an already active level by adding a validation check:

```
error LevelAlreadySubscribed();  
require(!isLevelActive(msg.sender, level), LevelAlreadySubscribed());
```

Resolution:

Fixed in [c9d413a](#). The protection against the re-subscription to the same level has been implemented in the `subscribeToLevel()` function as follows:

```
function subscribeToLevel(uint256 level) external validLevel(level) nonReent  
rant {  
    ...  
    if (isLevelActive(userId, level)) revert LevelAlreadySubscribed();  
    ...  
}
```

[F-2025-14483](#) - Second Stake Inherits Old Activity Time - Reduced Reward Window - Medium

Description:

When a user adds a second stake while already having an active stake, the `lastActivityTime` is **not** updated. This causes the second stake to inherit the old activity timestamp from the first stake, potentially leaving it with only a fraction of the expected 48-hour reward window before becoming "inactive."

Impact

Users lose significant rewards on their second stake:

1. User creates first stake at $T=0 \rightarrow \text{lastActivityTime} = 0$
2. User waits 47 hours without claiming
3. User creates second stake at $T=47h \rightarrow \text{lastActivityTime}$ stays at 0 (NOT updated)
4. Just 1 hour later ($T=48h$), BOTH stakes become "inactive"
5. The second stake only earned rewards for 1 hour instead of 48 hours

Financial impact example:

- User stakes 10,000 ACE as second stake at $T=47h$
- At 10% monthly rate, expected 48h earnings: ~66.7 ACE
- Actual earnings (1 hour): ~1.4 ACE
- **Loss:** ~65 ACE (98% of expected rewards)

User expectations violated:

- Users reasonably expect that actively engaging with the protocol (staking) would refresh their activity timer
- Instead, their second stake is penalized by the time since their last claim

StakingAndRewards.sol:

```
function stake(uint256 amount) external onlyActive nonReentrant {
    require(amount > 0, "Cannot stake 0");

    // ... validation logic ...

    uint256 slotIndex = _findAvailableSlot(msg.sender);
    uint256 stakeId = nextStakeId++;

    /* Update activity time only if no active stakes exist */
    if (!_hasAnyActiveStakes(msg.sender)) {
        lastActivityTime[msg.sender] = block.timestamp; // ! Only updates for
FIRST stake!
```

```

    }

    activeStakes[msg.sender][slotIndex] = Stake({
        // ...
    });

```

The comment explicitly states: "Update activity time only if no active stakes exist" - meaning second stakes do **NOT** get an updated activity time.

This is used in **reward calculations**:

```

function _calculateActiveTimePeriod(address user, uint256 /* unused */) private view returns (uint256 activeUntil) {
    uint256 currentTime = block.timestamp;

    /* MAX INACTIVE PERIOD LOGIC ONLY */
    uint256 lastActivity = lastActivityTime[user]; // ! Uses old timestamp
    uint256 inactivityEndTime = lastActivity > 0 ? lastActivity + MAX_INACTIVE_PERIOD : type(uint256).max;
    bool isActive = currentTime <= inactivityEndTime;

    /* Use inactivity cutoff if user is inactive, otherwise use current time */
    /
    activeUntil = isActive ? currentTime : inactivityEndTime;
}

```

Assets:

- StakingAndRewards.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Accepted

Classification

Impact Rate:	2/5
Likelihood Rate:	4/5
Exploitability:	Independent
Complexity:	Medium
Severity:	Medium

Recommendations

Remediation:

Update `lastActivityTime` whenever a user stakes, not just on their first stake:

```
function stake(uint256 amount) external onlyActive nonReentrant {
    require(amount > 0, "Cannot stake 0");

    // ... existing code ...

    uint256 slotIndex = _findAvailableSlot(msg.sender);
    uint256 stakeId = nextStakeId++;

    // FIXED: Always update activity time on stake
    lastActivityTime[msg.sender] = block.timestamp;

    activeStakes[msg.sender][slotIndex] = Stake({
        // ...
    });
}
```

Resolution:

The finding was **acknowledged** by the client with the following statement:

Design Intent: Activity-Based Reward System

The inactivity mechanism is designed to encourage regular engagement via claiming, not just staking.

How the Inactivity System Works:

1. First Stake Sets Starting Point:

- The first stake sets `lastActivityTime` as the baseline for reward calculation.
- This establishes when the user's activity window begins.

2. Only Claiming Resets Activity Timer:

- After the first stake, only `claimRewards()` updates `lastActivityTime`.
- Additional stakes do not reset the timer.
- This ensures users must actively claim to maintain their reward eligibility.

3. 48-Hour Inactivity Window:

- Users have 48 hours from their last claim to claim again.
- If they don't claim within 48 hours, rewards stop accruing at the inactivity cutoff.
- This window applies to all stakes, not per-stake.

Why This Design:

1. Encourages Regular Engagement:

- Users must claim at least once every 48 hours to keep earning.
- Prevents passive accumulation without active participation.

2. Fair Reward Distribution:

- Rewards are tied to active participation, not just initial staking.
- All users follow the same 48-hour claim requirement.

3. Protocol Sustainability:

- Prevents indefinite reward accrual without engagement.
- Ensures active users benefit from the reward pool.

Why Second Stake Doesn't Reset Timer:

1. Consistency: All stakes share the same activity status.
2. Engagement: Staking alone is not considered activity for reward purposes. First Stake only set the starting time for reward generation. Only claim is considered an activity.

User Expectations:

- Users are expected to claim at least once every 48 hours to maintain rewards.
- The system rewards active participants, not passive stakers.
- This is a feature, not a bug.

Conclusion:

This is intentional business logic to:

- Encourage regular claiming (at least once per 48 hours).
- Ensure active participation for reward eligibility.
- Maintain protocol sustainability through engagement requirements.

The behavior is:

- Deterministic and transparent in the code.
- Applied equally to all users.
- Designed to balance user rewards with protocol sustainability.

Evidences

PoC

Reproduce:

Run the following command with the PoC test file:

```
forge test --match-test test_SecondStakeBecomeInactiveImmediately -vv
```

Results:

Output:

```
[PASS] test_SecondStakeBecomeInactiveImmediately() (gas: 1223862)
```

Logs:

```
=== STEP 1: First Stake at Hour 0 ===
```

```
Created: 1,000 ACE stake
```

```
lastActivityTime set to: Hour 0

=== STEP 2: Second Stake at Hour 47 ===
Time elapsed: 47 hours
Time until inactivity: 1 hour remaining

Created: 1,000 ACE second stake
BUG: lastActivityTime NOT updated! Still at Hour 0

Current pending rewards: 6 ACE

=== STEP 3: Inactivity at Hour 48 ===
48 hours passed since Hour 0 (first stake)
User is now INACTIVE - rewards frozen!

Frozen pending rewards: 6 ACE

=== STEP 4: Verify Frozen at Hour 72 ===
Pending rewards (24h later): 6 ACE
```

Files:

SecondStakeInactivity.t.sol

[F-2026-14497](#) - Incorrect Referral Traversal in `_addDownlines()` Skips Intermediate Uplines - Medium

Description:

The `Subscription::_addDownlines()` function incorrectly traverses the referral hierarchy when propagating downline updates using `current.referrers[level]`.

During registration, `register()` first builds the referrer chain via `_buildReferrerChain()`, storing up to 12 upstream referrer IDs in `users[userId].referrers`. This array is ordered such that:

- `referrers[0]` → direct referrer (parent)
- `referrers[1]` → referrer of the parent
- etc.

The function `_addDownlines()` is then invoked to propagate the newly registered user across uplines' downline structures. The function advances as follows for traversing the tree:

```
current = users[current.referrers[level]];
```

which can be seen in the `_addDownlines()` function below:

```
function _addDownlines(uint256 newUserId, uint256 referrerId) private {
    User storage current = users[referrerId];

    // Cache constants for gas efficiency
    uint256 maxLevels = MAX_LEVELS;
    uint256 maxLevelsMinus1 = maxLevels - 1;

    for (uint256 level = 0; level < maxLevels; level++) {
        if(current.id == 0) break;

        /* Add to tier system instead of direct array push */
        addToTier(downlineTiers[current.id][level], newUserId);
        downlineCount[current.id][level]++;

        /* Update tier count if needed */
        uint256 newTierCount = getTierCount(downlineCount[current.id][level]);

        if (newTierCount > downlineTierCount[current.id][level]) {
            downlineTierCount[current.id][level] = newTierCount;
        }

        emit DownlineAdded(current.id, level, newUserId);
    }
}
```

```

        if (level < maxLevelsMinus1) {
            current = users[current.referrers[level]];
        } else {
            break;
        }
    }
}

```

This logic incorrectly jumps to the level-th ancestor of the current node, rather than progressing through the immediate upline (referrers[0]). As the level increases, this causes the traversal to skip intermediate ancestors, resulting in misplaced downline assignments and desynchronized per-level accounting.

For instance, for the built chain of referrers:

```

level 0 referrer 5004
level 1 referrer 5003
level 2 referrer 5002
level 3 referrer 5001
level 4 referrer 5000
level 5 referrer 1

```

the `_addDownlines()` loops them as follows:

```

5004
5003
5001

```

instead of

```

5004
5003
5002
5001
5000
1

```

This issue silently corrupts referral accounting and becomes increasingly severe as the referral depth grows.

Assets:

- Subscription.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Medium
Severity:	Medium

Recommendations

Remediation:

It is recommended to:

- always advance the traversal using the direct upline relationship:

```
current = users[current.referrers[0]];
```

- add invariant checks: ensure that per-level downline counts are consistent with aggregate totals after updates.
- add unit tests: Include multi-level referral scenarios that verify:
 - no ancestor is skipped
 - downline tiers align correctly with referral depth
 - per-level and total counters remain synchronized
- if the `downlineCount` mapping is not expected to be used, consider deleting it.

Resolution:

Fixed in commit `f9604f9`. `Subscription::_addDownlines()` was corrected to traverse the upline chain step-by-step by always advancing via the direct parent (`current.referrers[0]`) rather than jumping to `current.referrers[level]`.

Evidences

POC

Reproduce:

1. ALICE, BOB, CARLO, DAVID, EVE, FRANK, GEORGE, HARRY, IVY, JANE, KATE, LUCY register within a system using each other's userID as a referrerID.
2. After the latest registration, it is expected that the first registerId will have the non zero value on every level out of 12, which is not the case. It happens due to the incorrect increase of downline level.

Note: `downlineCount` mapping should be made public for the test to work.

Results:

```
function testDownlines() public {  
    vm.warp(1767352683);  
    vm.startPrank(ALICE);  
    usdt.approve(address(subscription), type(uint256).max);  
    subscription.register(1); // register under genesis  
    assertEq(subscription.getGenesisCount(), 1);  
    vm.stopPrank();  
  
    vm.startPrank(BOB);  
    usdt.approve(address(subscription), type(uint256).max);  
    subscription.register(5000); // register under ALICE  
    vm.stopPrank();  
  
    vm.startPrank(CARLO);  
    usdt.approve(address(subscription), type(uint256).max);  
    subscription.register(5001); // register under BOB  
    vm.stopPrank();  
  
    vm.startPrank(DAVID);  
    usdt.approve(address(subscription), type(uint256).max);  
    subscription.register(5002); // register under CARLO  
    vm.stopPrank();  
  
    vm.startPrank(EVE);  
    usdt.approve(address(subscription), type(uint256).max);  
    subscription.register(5003); // register under DAVID  
    vm.stopPrank();  
  
    vm.startPrank(FRANK);  
    usdt.approve(address(subscription), type(uint256).max);  
    subscription.register(5004); // register under EVE  
    vm.stopPrank();  
    ...  
}
```

Files:

Downlines.t.sol

[F-2026-14501](#) - Unstake Forfeits Accumulated Rewards Without Documentation/Notice - Medium

Description:

The `unstake()` function in `StakingAndRewards.sol` silently forfeits users' accumulated but unclaimed rewards. When a user unstakes, both their pending staking rewards and accumulated team rewards are lost without warning. This behavior is not documented and can lead to unexpected financial losses for users.

Impact

Users who call `unstake()` without first calling `claimRewards()` will lose:

1. **Pending Staking Rewards:** All accrued staking rewards that haven't been claimed are forfeited. Stakes are deactivated without paying out pending rewards.
2. **Accumulated Team Rewards:** All team rewards accumulated across 12 levels are explicitly deleted (set to 0).

This is particularly harmful because:

- The documentation only mentions a 20% penalty and deduction of already-claimed rewards
- Users may assume pending rewards will be paid out during unstake
- Team rewards can accumulate significantly over time and are completely lost
- There is no on-chain warning or event indicating rewards are being forfeited

Example Loss Scenario:

- User has 66 ACE in pending staking rewards
- User has 3 ACE in accumulated team rewards
- User unstakes directly → receives only 8,000 ACE (80% of stake)
- User loses 69 ACE in rewards they were entitled to

Team rewards deletion (`StakingAndRewards.sol`):

```
/* Clear team rewards efficiently */
uint256 maxLevels = 12;
for (uint256 level = 0; level < maxLevels; level++) {
    delete teamRewards[user][level];
}
```

Stake deactivation without pending reward payout (`StakingAndRewards.sol`):

```

if (stake0.active) {
    totalStakeAmount += stake0.amount;
    totalClaimedRewards += stake0.totalClaimed; // Only tracks ALREADY claimed
    stake0.active = false; // Deactivated - pending rewards lost!
}
if (stake1.active) {
    totalStakeAmount += stake1.amount;
    totalClaimedRewards += stake1.totalClaimed;
    stake1.active = false;
}

```

Documentation states:

A user may unstake/disable their stake at any time, subject to penalties:

--20% of the staked amount is deducted.

--Any rewards already claimed are also deducted.

--The user's wallet/account is permanently blocked from staking again in the future.

Rewards user previously called `claimRewards()` for is deducted from the returned amount:

```

int256 penalty = (totalStakeAmount * 20) / 100;
uint256 totalDeductions = penalty + totalClaimedRewards; // ← "deducted"
uint256 returnAmount = totalStakeAmount > totalDeductions ? totalStakeAmount
    - totalDeductions : 0;

```

However, **no mention** of pending (accrued) rewards or team rewards being forfeited.

Assets:

- StakingAndRewards.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Accepted

Classification

Impact Rate: 3/5

Likelihood Rate: 4/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation:

Clearly document that:

- Pending staking rewards are NOT paid during `unstake()`
- All accumulated team rewards are forfeited
- Users should call `claimRewards()` before `unstake()` to receive pending rewards

Alternatively, send accrued rewards to users during the `unstake()` call.

Resolution:

The finding was **acknowledged** by the client with the following statement:

Thank you for the detailed analysis of the `unstake()` behavior in `StakingAndRewards.sol`.

We would like to clarify that the current design of `unstake()` is intentional business logic, not an oversight in reward handling.

Design intent: `unstake()` is an exit-with-penalty path, not a claim path

The protocol defines two distinct user actions:

1. `claimRewards()` – used to realize all pending staking and team rewards.
2. `unstake()` – used to permanently exit staking with a penalty on principal and on already-claimed rewards.

The economic model is:

- Users who want to capture their rewards should call `claimRewards()` first.
- Users who choose to exit via `unstake()` accept that:
- Their principal is returned with a 20% penalty.
- Any rewards they have already realized are also deducted from the returned principal. So, Even users claim before `unstake` they would still lose those rewards.
- Any unrealized (pending) staking and team rewards are forfeited as part of the exit.

This is by design to make early/forced exit clearly less favorable than claiming regularly and staying in the system.

[F-2026-14510](#) - Unimplemented pendingReward State Variable Results in Dead Code and Potential Reward Accounting Issues - Medium

Description:

The `StakingAndRewards.sol` contract contains a `pendingReward` field in the `Stake` struct that is initialized to zero upon stake creation but is **never updated** throughout the contract's lifetime. This represents dead code that suggests an incomplete implementation of the reward accounting mechanism, potentially leading to loss of unclaimed rewards in edge cases and unnecessary gas costs.

The `pendingReward` field is defined in the `Stake` struct

```
struct Stake {  
    uint256 stakeId;  
    uint256 amount;  
    uint256 startTime;  
    uint256 lastClaimTime;  
    uint256 totalClaimed;  
    uint256 pendingReward;  
    bool active;  
}
```

and initialized in the `StakingAndRewards::stake()` function. During stake creation, this field is set to zero:

```
function stake(uint256 amount) external onlyActive nonReentrant {  
    ...  
    activeStakes[msg.sender][slotIndex] = Stake({  
        stakeId: stakeId,  
        amount: amount,  
        startTime: block.timestamp,  
        lastClaimTime: block.timestamp,  
        totalClaimed: 0,  
        pendingReward: 0,  
        active: true  
    });  
    ...  
}
```

The field is subsequently read in multiple reward calculation functions but never written to:

```
function _calculateRewardAmount(Stake storage stakeEntry, uint256 activeU
```

```

until, uint256 /* unused */, address /* unused */) private view returns (uint2
56 finalReward, bool shouldContinue) {
    uint256 startTime = stakeEntry.lastClaimTime;

    /* For first claim, use stake start time, otherwise use lastClaimTime
    */
    if (startTime == 0) {
        startTime = stakeEntry.startTime;
    }

    if (activeUntil <= startTime) {
        return (stakeEntry.pendingReward, false);
    }

    uint256 elapsed = activeUntil - startTime;
    uint256 rate = _getRewardRate();
    uint256 newReward = (stakeEntry.amount * rate * elapsed) / (MONTH * P
ERCENT_DENOMINATOR);
    uint256 totalReward = stakeEntry.pendingReward + newReward;

    uint256 maxReward = (stakeEntry.amount * MAX_REWARD_MULTIPLIER) / PER
CENT_DENOMINATOR;
    if (stakeEntry.totalClaimed >= maxReward) {
        return (0, false);
    }

    uint256 remainingReward = maxReward - stakeEntry.totalClaimed;
    finalReward = totalReward > remainingReward ? remainingReward : total
Reward;
    shouldContinue = false;
}

```

Similarly in `_calculateSingleReward()`:

```

function _calculateSingleReward(Stake storage stakeEntry, address user) priva
te view returns (uint256) {
    if (!stakeEntry.active) return 0;

    uint256 userId = affiliate.userIds(user);
    if (userId == 1) return 0;

    uint256 lastPaymentTime = affiliate.getLevelPaymentInfo(userId, 0);
    if (lastPaymentTime == 0) return 0;

    uint256 activeUntil = _calculateActiveTimePeriod(user, userId);
    uint256 startTime = stakeEntry.lastClaimTime;

    if (startTime == 0) {

```

```

        startTime = stakeEntry.startTime;
    }

    if (activeUntil <= startTime) {
        return stakeEntry.pendingReward;
    }

    uint256 elapsed = activeUntil - startTime;
    uint256 rate = _getRewardRate();
    uint256 newReward = (stakeEntry.amount * rate * elapsed) / (MONTH * P
    ERCENT_DENOMINATOR);
    uint256 totalReward = stakeEntry.pendingReward + newReward;

    uint256 maxReward = (stakeEntry.amount * MAX_REWARD_MULTIPLIER) / PER
    CENT_DENOMINATOR;
    if (stakeEntry.totalClaimed >= maxReward) return 0;

    uint256 remainingReward = maxReward - stakeEntry.totalClaimed;
    uint256 finalReward = totalReward > remainingReward ? remainingReward
    : totalReward;

    return finalReward;
}

```

Impact:

1. **Reward Loss in Edge Cases:** When `activeUntil <= startTime` (indicating an inactive period), the contract returns `stakeEntry.pendingReward` which is always 0. This means any rewards that should have been pending from previous active periods are effectively lost, as they were never stored in `pendingReward`.
2. **Gas Inefficiency:** Every Stake struct wastes 32 bytes of storage (one storage slot) for a field that serves no purpose, increasing deployment and operational costs unnecessarily.
3. **Code Maintainability:** The presence of unused code suggests incomplete implementation and makes the codebase harder to audit and maintain, potentially masking other vulnerabilities.
4. **Design Inconsistency:** The code structure implies that rewards should accumulate in `pendingReward` during activity gaps, but this mechanism is non-functional, creating a gap between intended and actual behavior.

Assets:

- StakingAndRewards.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: It is recommended to:

- consider the system redesign and implementation of the proper pendingReward tracking.
- remove the dead unused code.

Resolution: **Fixed** in `1829f31`. The redundant `pendingReward` variable was removed from the codebase.

F-2025-14444 - Missing SafeERC20 For Not ERC20 Compliant USDT - Low

Description:

The `Subscription` contract performs direct ERC20 interactions with the USDT token using the raw ERC20 interface and assumes full ERC20 compliance, including boolean return values and strict revert behavior. However, **USDT is a well-known non-standard ERC20 token**, and relying on raw calls introduces fragile and unsafe payment logic. This pattern appears in multiple user-facing functions, including `register()`, `renewSubscription()`, and `subscribeToLevel()`, where balance and allowance checks are followed by a direct `transferFrom()` call.

While the contract explicitly checks balances and allowances beforehand, these checks **do not guarantee successful transfers** for non-compliant ERC20 implementations. USDT historically exhibits inconsistent behavior across deployments (e.g., not returning a boolean, silently succeeding, or reverting unexpectedly), which can cause unexpected transaction failures or inconsistent state assumptions.

The affected logic relies on the following pattern:

```
...
require(usdtToken.balanceOf(msg.sender) >= fee, "Insufficient USDT balance");
require(usdtToken.allowance(msg.sender, address(this)) >= fee, "Insufficient
USDT allowance");
require(usdtToken.transferFrom(msg.sender, aceFoundationWallet, fee), "ERC20:
transfer failed");
...
```

Assets:

- Subscription.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple

Severity:

Low

Recommendations**Remediation:**

It is recommended to:

- replace all raw ERC20 calls with OpenZeppelin's SafeERC20 library as follows:

```
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol"

...

using SafeERC20 for IERC20;

...

usdtToken.safeTransferFrom(msg.sender, aceFoundationWallet, fee);

...
```

- remove redundant `balanceOf()` and `allowance()` pre-checks, as SafeERC20 already enforces safe execution and revert behavior.
- standardize ERC20 handling across the entire codebase to avoid inconsistent token interaction patterns.

Adopting SafeERC20 aligns the protocol with **industry best practices** and significantly improves robustness, forward compatibility, and production safety.

Resolution:

Fixed in `f9604f9`. The SafeERC20 library was implemented in the functions of the `Subscription` contract.

[F-2025-14465](#) - `getSubscribedLevels()` Returns Expired Levels as Active - Low

Description:

The `getSubscribedLevels()` function only checks if `levelInfo[i].isActive` is `true` but does NOT verify if the level subscription has expired. This causes the function to return expired levels as "active", leading to incorrect data for frontends, integrations, and analytics.

Vulnerable Function:

- `Subscription.sol::getSubscribedLevels()`

Subscription.sol:

```
function getSubscribedLevels(uint256 userId) external view validUserId(userId)
returns (uint256[] memory) {
    // Cache storage reads and constants for gas efficiency
    User storage user = users[userId];
    uint256 maxLevels = MAX_LEVELS;

    uint256 activeCount = 0;
    for (uint256 i = 0; i < maxLevels; i++) {
        if (user.levelInfo[i].isActive) { // ! Only checks isActive flag
            activeCount++;                // Does NOT check expiry time!
        }
    }

    uint256[] memory activeLevels = new uint256[](activeCount);
    uint256 index = 0;
    for (uint256 i = 0; i < maxLevels; i++) {
        if (user.levelInfo[i].isActive) { // ! Same issue in second loop
            activeLevels[index] = i;
            index++;
        }
    }

    return activeLevels;
}
```

The function is **inconsistent** with `isLevelActive()`:

```
// After expiry:
isLevelActive(userId, 0) → false    Correct
getSubscribedLevels(userId) → [0]   Incorrect (claims level 0 is active)
```

This creates two sources of truth that contradict each other.

Assets:

- Subscription.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation:

Consider using `isLevelActive()` for an accurate check:

```
function getSubscribedLevels(uint256 userId) external view validUserId(userId) returns (uint256[] memory) {
    User storage user = users[userId];
    uint256 maxLevels = MAX_LEVELS;

    // First pass: Count actually active levels
    uint256 activeCount = 0;
    for (uint256 i = 0; i < maxLevels; i++) {
        if (isLevelActive(userId, i)) { // ← FIX: Use isLevelActive() instead
            activeCount++;
        }
    }

    // Second pass: Populate array with active levels
    uint256[] memory activeLevels = new uint256[](activeCount);
    uint256 index = 0;
    for (uint256 i = 0; i < maxLevels; i++) {
        if (isLevelActive(userId, i)) { // ← FIX: Use isLevelActive() instead
            activeLevels[index] = i;
            index++;
        }
    }
}
```

```
return activeLevels;  
}
```

Resolution:

Fixed in [f9604f9](#). The function `getSubscribedLevels()` was modified and now properly returns the active subscribed levels using the function `isLevelActive()`.

F-2025-14467 - Missing Minimum Fee Validation in `setDefaultFee()` - Low

Description:

The `setDefaultFee()` function validates the maximum fee (3 USDT) but does not enforce a minimum fee value. This allows the admin to accidentally set the fee to zero, which would cause all subscriptions to become free.

While this has no direct security implications or profit motive for attackers, it represents a configuration risk that could impact platform revenue if triggered by mistake.

Affected Function:

- `Subscription.sol::setDefaultFee()`

If the admin accidentally sets the fee to zero:

- Users can register without payment
- Subscription renewals require no payment
- Level subscriptions are free
- The platform temporarily operates without subscription revenue

Subscription.sol:

```
function setDefaultFee(uint256 fee) external onlyOwner {
    require(fee <= MAX_FEE, "Exceeds max fee"); ! Only validates maximum
    require(fee != defaultFee, "Fee unchanged");
    defaultFee = fee;
    _setAllLevelFees(fee);
    emit FeeUpdated(address(0), fee);
}
```

Assets:

- Subscription.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 4/5

Exploitability: Dependent

Complexity: Simple

Severity:

Low

Recommendations

Remediation:

It is recommended to implement a lower bound for the minimum configurable fee value to prevent zero fees.

Resolution:

Fixed in `f9604f9`. The lower bound for the minimum configurable default fee value set to 1 USDT or 1e6 to prevent the fees from being zero.

[F-2025-14476](#) - Missing Maximum Fee Cap in setCustomFee() Allows Arbitrary Fee Assignment - Low

Description:

The `setCustomFee()` function allows the contract owner to assign a custom subscription fee to any user without enforcing an upper bound. Unlike `setDefaultFee()`, which explicitly restricts fees using the `MAX_FEE` constant, `setCustomFee()` lacks any validation to ensure the assigned fee remains within acceptable limits.

```
function setCustomFee(address user, uint256 fee) external onlyOwner validAddress(user) {
    require(fee != customFees[user], "Fee unchanged");
    customFees[user] = fee;
    hasCustomFee[user] = true;
    _setCustomAllLevelFees(user, fee);
    customFeeChangeTimestamp[user] = block.timestamp;
    emit FeeUpdated(user, fee);
}
```

This inconsistency allows the owner to set arbitrarily large fees (e.g., exceeding `MAX_FEE`), bypassing the intended economic constraints enforced elsewhere in the protocol.

Assets:

- Subscription.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation:

It is recommended to:

- Enforce the same maximum fee constraint used for default fees by validating the input against `MAX_FEE`:

```
require(fee <= MAX_FEE, "Exceeds max fee");
```

- Explicitly documenting whether custom fees are intended to bypass protocol-wide economic limits.
- Introducing a time-delay or governance mechanism for fee changes to reduce abuse risk.

Resolution:

Fixed in commit `f9604f9`. A fee-range validation was added to `setCustomFee()` by introducing `MAX_CUSTOM_FEE` and enforcing `MIN_FEE <= fee <= MAX_CUSTOM_FEE`, preventing arbitrary custom-fee assignments beyond the protocol's intended economic bounds while preserving the existing tighter cap for `setDefaultFee()`.

F-2026-14502 - Team Rewards Exceeding Daily Limit Are Permanently Deleted - Low

Description:

The `StakingAndRewards.sol` contract implements a daily cap on team rewards (`MAX_DAILY_TEAM_REWARD = 30 ACE`). However, the implementation of this cap is flawed: when a user claims team rewards, any amount exceeding the daily limit is permanently deleted from storage instead of being carried forward to the next day.

Impact

Referrers lose any team rewards that accumulate beyond 30 ACE in a single day (or between claims).

For example:

1. A referrer has 500 ACE in accumulated team rewards from various downline claims.
2. The referrer calls `claimRewards()`.
3. The contract calculates that the referrer is eligible for 30 ACE (the daily limit).
4. The contract **deletes all 500 ACE** from the referrer's storage, pays out 30 ACE, and effectively voids the remaining 470 ACE.

This leads to a significant loss of funds for active referrers with large teams, as they are entitled to 5% of all downline claims but are prevented from receiving them if they don't claim extremely frequently, and even then, any single large claim from a downline that generates >30 ACE in team rewards will result in immediate forfeiture of the surplus.

In `_calculateTeamRewards()`, the contract calculates the total available rewards but then clears all of them regardless of the claim amount.

StakingAndRewards.sol:

```
/* Calculate total available team rewards */
uint256 totalAvailable = 0;
for (uint256 level = 0; level < 12; level++) {
    totalAvailable += teamRewards[user][level].totalReward;
}

/* Claim amount is minimum of available rewards and remaining daily limit */
totalClaimable = totalAvailable < (MAX_DAILY_TEAM_REWARD - alreadyClaimedToday) ? totalAvailable : (MAX_DAILY_TEAM_REWARD - alreadyClaimedToday);

if (totalClaimable > 0) {
    /* Clear all accumulated rewards (no carry forward) */
```

```

    for (uint256 level = 0; level < 12; level++) {
        teamRewards[user][level].totalReward = 0; // ! Surplus is deleted here
    }
    /* Update daily claimed amount */
    dailyTeamRewardsClaimed[user][currentDay] += totalClaimable;
}

```

Assets:

- StakingAndRewards.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Accepted

Classification

Impact Rate:	2/5
Likelihood Rate:	4/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation: Modify `_calculateTeamRewards` to only deduct the amount actually being claimed from the storage, rather than deleting everything. However, since rewards are stored per level, a FIFO or proportional deduction strategy is needed, or the storage structure should be simplified to a single `totalTeamReward` variable per user.

Resolution: The finding was **acknowledged** by the client with the following statement:

We would like to clarify that this behavior is intentional business logic, not an unintended vulnerability.

Design intent of the cap

The `'MAX_DAILY_TEAM_REWARD = 30e18'` (30 ACE) limit is meant to be a **hard daily cap** on team rewards that a referrer can realize per day. Any team rewards generated beyond this cap are **not meant to be carried forward**; they are intentionally forfeited. In other words, the system is designed such that a referrer can receive **up to** 30 ACE per day from team rewards, and no more, regardless of how large their team is or how much volume is generated.

This non-carry-forward model was chosen:

- To strictly control long-term token emissions and protect the reward pool and protocol sustainability.
- To prevent unbounded accumulation of liabilities from very large teams over long periods.
- To encourage regular engagement/claiming behavior within a predictable daily limit rather than guaranteeing full accrual of all mathematically possible team rewards.

On the "loss" of surplus rewards:

From our perspective, the "extra" team rewards above 30 ACE per day are **not considered guaranteed entitlements**, but rather **potential rewards subject to the daily cap**. The logic in `_calculateTeamRewards()` that clears all accumulated rewards after applying the daily cap is the deliberate mechanism to enforce this economic rule:

- The referrer's **maximum realizable** team reward per day is 30 ACE.
- Any excess is intentionally not preserved for future days.

Security vs. economics:

We agree that this behavior can be surprising if one assumes all accumulated team rewards should be fully claimable over time. However, in our design this is a business/economic constraint not a security flaw:

- There is no unintended loss of user-deposited funds or staked principal.
- The behavior is deterministic, transparent in the code, and applies equally to all participants.
- It aligns with the protocol's chosen tokenomics and emission policy.

Evidences

PoC

Reproduce:

Run the following command to execute the PoC file:

```
forge test --match-contract TeamRewardForfeitureValidationTest -vv
```

Results:

Output:

```
[PASS] test_SurplusTeamRewardsAreDeleted() (gas: 2010650)
```

Logs:

```
===Validation: Surplus Team Rewards Deletion ===
```

```
Timestamp: 1735689600 (Jan 1, 2026)
```

```
Step 1: Referrer staked 10,000 ACE
Step 2: Staker staked 1,000,000 ACE

Step 3: Warped 2 days forward
Staker pending rewards: 6666 ACE
Staker claimed successfully

Step 4: Referrer's accumulated team rewards BEFORE claim:
Level 0 team rewards: 333 ACE
Total accumulated: 333 ACE
Daily limit: 30 ACE

Does accumulated exceed daily limit? true
Surplus amount: 303 ACE

Step 5: Referrer claimed rewards:
Balance before: 100000 ACE
Balance after: 100096 ACE
Total received: 96 ACE

Step 6: Referrer's team rewards AFTER claim:
Level 0 team rewards: 0 ACE
Total remaining: 0 ACE

=== ANALYSIS ===
Accumulated before claim: 333 ACE
Daily limit (max payable): 30 ACE
Remaining after claim: 0 ACE

EXPECTED remaining (if preserved): 303 ACE
ACTUAL remaining: 0 ACE

*** BUG CONFIRMED ***
Surplus rewards were DELETED, not preserved!
```

Files:

TeamRewardForfeitureValidation.t.sol

[F-2026-14506](#) - Incorrect Boundary Condition Activates Protection System Despite Sufficient Contract Balance - Low

Description:

The `StakingAndRewards::isProtectionSystemActive()` function contains an **incorrect comparison operator** when evaluating whether the protection system should be activated. The function uses a less-than-or-equal (`<=`) check instead of a strictly less-than (`<`) comparison, causing the protection system to activate **even when the contract holds exactly the required funds** to honor all guaranteed rewards.

The protection system is designed to activate only when the contract balance is insufficient to cover all outstanding obligations. The required threshold is calculated as:

```
threshold = totalActiveStaked * PROTECTION_THRESHOLD_MULTIPLIER
```

With `PROTECTION_THRESHOLD_MULTIPLIER = 2`, this threshold represents the **maximum payout needed to honor all 2* reward guarantees**.

However, the current implementation performs the following check:

```
function isProtectionSystemActive() internal view returns (bool) {
    uint256 totalActive = getTotalActiveStakes();
    uint256 threshold = totalActive * PROTECTION_THRESHOLD_MULTIPLIER;
    uint256 contractBalance = IERC20(aceToken).balanceOf(address(this));
    return contractBalance <= threshold;
}
```

This logic activates the protection system not only when the contract balance is **below** the required threshold, but also when it is **exactly equal** to it.

Mathematically, this is incorrect:

If `contractBalance == threshold`, the system has precisely enough funds to fulfill all obligations. Protection should only activate when `contractBalance < threshold`.

Although this issue does not directly affect user balances, it has meaningful protocol-level consequences:

- **False-Positive Protection Activation:**

The protection system can be triggered even when the contract is fully solvent.

- **Unnecessary Restriction of Core Functionality:**

Legitimate staking, unstaking, or reward operations may be blocked or altered prematurely.

- **Incorrect Risk Signaling:**

Frontend dashboards or monitoring systems may incorrectly signal insolvency.

- **Reduced Capital Efficiency:**

Funds are treated as insufficient when they are actually adequate, degrading protocol usability.

Assets:

- StakingAndRewards.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Medium

Severity: Low

Recommendations

Remediation:

It is recommended to:

- replace the non-strict comparison with a strictly less-than check as follows:

```
function isProtectionSystemActive() internal view returns (bool) {
    uint256 totalActive = getTotalActiveStakes();
    uint256 threshold = totalActive * PROTECTION_THRESHOLD_MULTIPLIER;
    uint256 contractBalance = IERC20(aceToken).balanceOf(address(this));
    return contractBalance < threshold;
}
```

This ensures the protection system activates **only when the contract balance is truly insufficient**, preserving correct economic and risk semantics.

- add inline documentation clarifying the solvency invariant
- add unit tests covering boundary conditions

Resolution:

Fixed in commit [f9604f9](#). The protection-system activation check in `isProtectionSystemActive()` was corrected from `<=` to a strict `<`, preventing

false-positive activation when the contract balance exactly equals the solvency threshold.

F-2025-14427 - Floating Pragma - Info

Description:

The contracts in this project use a "floating pragma" declaration (pragma solidity ^0.8.20), allowing the code to be compiled with any compiler version equal to or above 0.8.20. While a floating pragma offers flexibility during development, it can introduce significant risks for deployment, particularly due to potential breaking changes in future Solidity versions, compatibility issues with non-Ethereum chains, and vulnerability to Solidity compiler bugs.

New Solidity versions may introduce changes, deprecate features, or contain bugs that could result in unintended behavior in previously deployed contracts. Key concerns include:

- **Breaking Changes in Future Versions:** Solidity updates occasionally introduce syntax changes, new features, or optimizations that could alter contract behavior if compiled with newer versions. This variability may lead to unexpected results or make the contract vulnerable to issues that were not present in the version originally used during development and testing.
- **Solidity Compiler Bugs:** Solidity frequently updates its bug documentation, and known issues for each version are documented in the [Solidity documentation](#). Using a floating pragma risks exposing the contract to new bugs introduced in future compiler versions.
- **Opcode Support and Cross-Chain Compatibility:** Solidity 0.8.20 and higher introduced new opcodes, supported on Ethereum mainnet but not universally across Layer 2 (L2) chains or alternative networks. Deploying contracts with floating pragmas on these networks could lead to compatibility issues or execution failures due to unsupported opcodes. Checking opcode compatibility across chains is possible using [evmdiff](#), but a locked pragma with a widely supported compiler version can help avoid these issues.

Using a floating pragma can lead to the following issues:

- **Stability Risks:** Contracts compiled with newer compiler versions may behave unpredictably due to untested changes, affecting stability.
- **Cross-Chain Compatibility:** Opcode differences in newer compiler versions may cause deployment failures or operational errors on networks lacking opcode support.
- **Security Risks:** Floating pragmas leave contracts susceptible to unforeseen vulnerabilities or bugs introduced in newer Solidity versions.

Assets:

- StakingAndRewards.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

- Subscription.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]
- MemoryManager.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]
- AceCoin.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

It is recommended to:

- replace all floating pragmas with a fixed Solidity version, after fully testing and reviewing compatibility with the target deployment networks.
- verify that the selected version supports the necessary opcodes across networks using [evmdiff](#) and review known bugs for this version on the [Solidity official site](#) to ensure stability. Select a Solidity version that is EVM-compatible and widely supported on the intended deployment chains.

Resolution:

Fixed in [f9604f9](#): The floating pragma was fixed to 0.8.20 in all of the aforementioned contracts.

[F-2025-14428](#) - Custom Ownership Implementation Causes Weaker Access Control Guarantees - Info

Description:

The `Subscription` and `StakingAndRewards` smart contracts introduce a **custom ownership mechanism** that reimplements the logic of OpenZeppelin's widely adopted `Ownable` contract instead of importing and inheriting from the official, audited implementation.

```
function transferOwnership(address newOwner) external onlyOwner {
    require(newOwner != address(0), "New owner is zero address");
    require(newOwner != owner, "New owner is same as current");
    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
}

function renounceOwnership() external onlyOwner {
    emit OwnershipTransferred(owner, address(0));
    owner = address(0);
}
```

Duplicating ownership logic increases the protocol's **attack surface and maintenance risk**. OpenZeppelin's access-control contracts have undergone extensive security reviews, real - world battle testing, and continuous improvements across multiple major releases. By reimplementing this logic internally, the protocol forfeits these guarantees and becomes solely responsible for maintaining correctness, compatibility, and security over time.

Assets:

- `StakingAndRewards.sol` [<https://github.com/AMGAceToken/Ace-SmartContracts>]
- `Subscription.sol` [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity:

Info

Recommendations

Remediation:

To strengthen access control guarantees and align the protocol with established industry best practices, it is recommended to **remove all custom ownership implementations** from the `Subscription` and `StakingAndRewards` contracts and instead use OpenZeppelin's standardized access-control `Ownable` modules.

Resolution:

Fixed in `f9604f9`. The custom `Ownable` pattern was successfully replaced with the functionality of the `Ownable2Step.sol` contract in `Subscription` and `StakingAndRewards` contracts.

F-2025-14430 - Potential Loss of Ownership Control in Contract Using Ownable - Info

Description:

The `Ownable` pattern, currently utilized by `StakingAndRewards` and `Subscription` contracts uses a single-step process for transferring ownership, as defined in the `transferOwnership()` function.

```
function transferOwnership(address newOwner) external onlyOwner validAddress
(newOwner) {
    require(newOwner != owner, "New owner is same as current");
    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
}
```

This single-step approach allows ownership to be transferred immediately to a new address, including potentially incorrect or invalid addresses (e.g., due to a typo). If the contract ownership is mistakenly transferred to an address that cannot manage it (such as a non-existent address or a contract without an appropriate `onlyOwner` function), it could result in permanent loss of control over the contract's critical functions.

The `Ownable` pattern is important for:

- Transferring and renouncing ownership
- Updating the foundation wallet address in the `StakingAndRewards` and `Subscription` smart contracts

Losing ownership would prevent the rightful owner from executing the previously mentioned operations.

Assets:

- `StakingAndRewards.sol` [<https://github.com/AMGAceToken/Ace-SmartContracts>]
- `Subscription.sol` [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation: To prevent accidental ownership transfers, consider using [Ownable2Step](#) from OpenZeppelin. This contract requires the new owner to confirm the transfer, reducing the risk of transferring ownership to an invalid address: Replace the custom Ownable pattern with functionality of the `Ownable2Step.sol` contract in `Subscription` and `StakingAndRewards` contracts.

Resolution: **Fixed** in `f9604f9`. The `Ownable` pattern was successfully replaced with the functionality of the `Ownable2Step.sol` contract in `Subscription` and `StakingAndRewards` contracts.

F-2025-14431 - Use Of Custom Errors Instead Of Revert Strings - Info

Description:

Starting with Solidity version 0.8.4, a new feature called "custom errors" was introduced. This allows developers to define specific error conditions in a clearer, more meaningful way without the need for string-based error messages. Before this version, developers commonly used `require` statements with strings to signal validation failures or specific conditions. However, including unique strings for revert reasons adds to gas costs, making transactions more expensive. For instance, the function `register()` uses unique strings for revert reasons:

```
function register(uint256 referrerId) external nonReentrant {
    require(userIds[msg.sender] == 0, "Already registered");
    require(referrerId < nextUserId, "Invalid referrer");
    if(referrerId == GENESIS_ID) {
        require(genesisCount == 0, "Genesis ID can only have 1 direct downline");
        genesisCount++;
        emit GenesisCountUpdated(genesisCount - 1, genesisCount);
    }
    User storage referrer = users[referrerId];
    require(referrer.userAddress != address(0), "Referrer doesn't exist");
    ;

    uint256 fee = getRegistrationFee(msg.sender);
    if (fee > 0 && aceFoundationWallet != address(0)) {
        require(usdtToken.balanceOf(msg.sender) >= fee, "Insufficient USD T balance");
        require(usdtToken.allowance(msg.sender, address(this)) >= fee, "Insufficient USDT allowance");
        require(usdtToken.transferFrom(msg.sender, aceFoundationWallet, fee), "ERC20: transfer failed");
    }
    ...
}
```

In contrast, custom errors are defined by their name and the types of their parameters, without the need for string data storage. As a result, using custom errors instead of string-based `require` messages reduces gas consumption, making contracts more efficient.

Assets:

- StakingAndRewards.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

- Subscription.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]
- MemoryManager.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

To enhance gas efficiency the code, it is recommended to consider replacing traditional require statements with custom errors. Custom errors are more cost-effective because they reduce the amount of data that needs to be stored and processed on the blockchain when handling exceptions. While require statements generate error messages stored as strings, which consume additional gas, custom errors are more compact and can save a significant amount of gas, especially in complex contracts with numerous validation checks. By using custom errors, a developer can streamline error handling and make the contracts more economical to operate without sacrificing clarity in identifying issues. For example:

```
...
error InvalidUserId();
error InvalidUserAddress();

modifier validUserId(uint256 userId) {
    require(userId != 0, InvalidUserId());
    require(users[userId].userAddress != address(0), InvalidUserAddress()
);
    _;
}
...
```

Resolution:

Fixed in [c9d413a](#). Gas inefficient string error messages have been replaced with the custom errors within the codebase

F-2025-14434 - Redundant Utility Functions Introduce Dead Code - Info

Description: The `MemoryManager` contract contains a helper function, `getTierAndIndex()`, which is currently unused across the entire codebase and partially redundant in its implementation:

```
function getTierAndIndex(uint256 globalIndex) internal pure returns (uint256
tier, uint256 index) {
    tier = globalIndex / TIER_SIZE;
    index = globalIndex % TIER_SIZE;
}
```

Other unused/dead functionality:

`Subscription::setUSDTAddress()`

```
function setUSDTAddress(address /* _newUSDTAddress */) internal pure {
    revert("USDT address is immutable");
}
```

While this issue does not introduce a direct security vulnerability, it has several negative side effects:

- **Maintenance risk:** future contributors may mistakenly assume the function is active or required, leading to incorrect integrations.
- **Codebase complexity:** unused helpers obscure the true execution paths of the protocol.
- **Gas and bytecode bloat:** unnecessary logic increases deployment size without providing functionality.

In security-critical systems, minimizing unused logic is considered a best practice, as dead code can hide future bugs or become incorrectly relied upon during refactors.

Assets:

- Subscription.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]
- MemoryManager.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:	It is recommended to: • remove the <code>getTierAndIndex()</code> and <code>setUSDTAddress()</code> functions entirely if it is not required by the current or planned design. • or consider utilizing them otherwise.
Resolution:	Fixed in commit <code>f9604f9</code>. Unused functions <code>getTierAndIndex()</code> (in <code>MemoryManager.sol</code>) and <code>setUSDTAddress()</code> (in <code>Subscription.sol</code>) were removed to reduce bytecode/deployment costs and avoid maintenance and integration confusion caused by dead or redundant helpers.

F-2025-14435 - Unused State Variables Introduce Security Risks - Info

Description:

The `StakingAndRewards` contract declares multiple **state variables that are never read from or written to in any execution path**. It creates **false assumptions** about protocol invariants and protections. The following variables `PROTECTION_MULTIPLIER`, `downlineCount` and `downlineTiers` are unused within the system. They suggest the existence of additional reward-protection logic and tier-based downline accounting. However, since these variables are never read from or written to, **the implied security mechanisms do not actually exist**.

This discrepancy between **declared state and enforced logic** can:

- mislead auditors, integrators, and future maintainers into believing that certain protections or accounting constraints are active
- cause incorrect assumptions during protocol upgrades, emergency responses, or external integrations
- lead to the introduction of unsafe changes when future logic is built on top of dormant storage variables
- mask incomplete or abandoned security features that were intended to mitigate financial risk

The presence of dormant variables increases the likelihood of human error, especially when ownership, staking limits, reward protection, or referral accounting is involved.

Assets:

- `StakingAndRewards.sol` [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

It is recommended to:

- remove all unused state variables and constants that are not required for the current protocol design.
- if these variables are intended for **future features**, clearly document this intention using comments or implement feature flags to avoid ambiguity.
- consider rerunning static analysis tools (e.g. Slither) to ensure that all state variables are utilized.

Resolution:

Fixed in commit `f9604f9`. Unused state variables (`PROTECTION_MULTIPLIER`, `downlineTiers`, and `downlineCount`) were removed from `StakingAndRewards`, eliminating misleading “implied” protections/accounting, reducing maintenance risk, and avoiding integrator/auditor confusion around non-existent invariants.

[F-2025-14455](#) - Stale userStakeTierCount After Unstake Breaks Tier Accounting Invariants - Info

Description: The `StakingAndRewards` contract maintains a `userStakeTierCount` variable to track how many stake tiers a user occupies:

```
mapping(address => uint256) public userStakeTierCount;
```

This value is updated when a user stakes,

```
...  
if (newTierCount > userStakeTierCount[msg.sender]) {  
    userStakeTierCount[msg.sender] = newTierCount;  
}  
...
```

But it is never updated during the unstake flow, causing the system to retain stale and incorrect tier metadata after a user exits.

As a result, the protocol assumes the user continues to occupy tiers that they no longer participate in, violating internal accounting assumptions and potentially breaking tier-based logic.

This creates a persistent inconsistency between:

- the user's actual staking state, and
- the protocol's tier-based bookkeeping.

Assets:

- `StakingAndRewards.sol` [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

It is recommended to decrease the `userStakeTierCount` on full unstake as follows:

```
...  
uint256 newTierCount = getTierCount(userStakeCount[msg.sender]);  
if (newTierCount > userStakeTierCount[msg.sender]) {  
    userStakeTierCount[msg.sender] = newTierCount;  
}  
...
```

Resolution:

Fixed in `c9d413a`. The affected `userStakeTierCount` state variable has been removed from the codebase.

[F-2025-14466](#) - `getUserCount()` Returns Misleading Value - Info

Description:

The `getUserCount()` function returns `nextUserId` (which starts at 5000) instead of the actual number of registered users. The function name suggests it returns a count, but it actually returns an internal ID counter, which can cause confusion in integrations and analytics.

Vulnerable Function:

- `Subscription.sol::getUserCount()`

Function name: `getUserCount()` → Implies it returns the count of users

Actual behavior: Returns `nextUserId` → An internal counter starting at 5000

Result: Integration confusion and incorrect analytics

Subscription.sol:

```
function getUserCount() external view returns (uint256) {  
    return nextUserId; // ! Returns 5000+ instead of actual count  
}
```

Assets:

- Subscription.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Return Actual Count:

```
/* Get total registered users */  
function getUserCount() external view returns (uint256) {  
    // Account for genesis user at ID 1 and regular users starting at 5000  
}
```

```
return (nextUserId - 5000) + 1; // +1 for genesis user  
}
```

Resolution:

Fixed in commit `f9604f9`. The codebase updated `getUserCount()` to return the actual registered user count (instead of the `nextUserId` counter) and adjusted `getContractStatistics()` to consume this corrected value directly.

F-2025-14486 - Lack of Public Access to Critical Protocol Data - Info

Description:

Both `StakingAndRewards` and the `Subscription` contracts maintain private or internal mappings that are actively mutated during protocol operation, but cannot be queried externally due to the lack of public access or external getter functions.

Affected Functionality:

`StakingAndRewards`:

- `lastStakeAmount`
- `lastClaimTime`
- `lastActivityTime`
- `hasUnstaked`
- `totalDirectRewards`
- `userStakeTierCount`

`Subscription`:

- `levelFees`
- `customLevelFees`
- `downlineTiers`
- `downlineCount`
- `downlineTierCount`

While the lack of public getters does not directly introduce an exploit, it creates significant operational and security risk:

- **Loss of transparency:** Users cannot verify downline tiers, or derived rewards that depend on this data.
- **Undetectable logic errors:** Bugs such as partial overwrites, truncated arrays, or incorrect indexing (e.g., exceeding fixed-depth limits) may silently corrupt state.
- **Auditability degradation:** External monitoring, indexing, and forensic analysis are severely limited, especially post-incident.
- **Governance and trust risk:** In systems with referral-based incentives, an opaque state undermines user trust and makes dispute resolution difficult.

Assets:

- `StakingAndRewards.sol` [<https://github.com/AMGAceToken/Ace-SmartContracts>]
- `Subscription.sol` [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:	It is recommended to provide a public access to the previously mentioned data by either adding a public access visibility or introducing separate getter functions.
Resolution:	Fixed in <code>c9d413a</code> . The necessary state variables have become available via separate getter functions.

[F-2025-14488](#) - Duplicate lastStakeAmount Assignment in stake() - Info

Description: In the `stake()` function, the `lastStakeAmount[msg.sender]` mapping is assigned the same value twice - at line 252 and again at line 261. The second assignment is redundant dead code that wastes gas.

Impact

- **Gas Waste** ~200 gas wasted per stake operation
- **Code Quality:** Redundant code reduces readability and maintainability
- **No Functional Impact:** The internal functions between the two assignments do not read or modify `lastStakeAmount`

StakingAndRewards.sol:

```
IERC20(aceToken).transferFrom(msg.sender, address(this), amount);
lastStakeAmount[msg.sender] = amount; // First assignment
totalStaked += amount;
totalActiveStaked += amount;
_processDirectReward(msg.sender, amount);
// Update level stake counters for all uplines
_updateLevelStakeCounters(msg.sender, amount, true);
lastStakeAmount[msg.sender] = amount; // Second assignment (REDUNDANT)
```

Assets:

- StakingAndRewards.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Remove the duplicate assignment to optimize gas usage.

Resolution:

Fixed in commit `f9604f9`. The codebase removed the redundant second assignment to `lastStakeAmount[msg.sender]` in `stake()`, reducing unnecessary gas usage and improving code clarity without changing behavior.

[F-2026-14503](#) - Precision Loss in Penalty Calculation Allows Penalty-Free Stake And Unstake for Small Amounts - Info

Description:

The `unstake()` logic suffers from an **integer precision loss** when calculating the unstaking penalty. Due to Solidity's integer division behavior, the computed penalty becomes **zero for small stake amounts**, allowing users to stake and immediately unstake without incurring any penalty. This breaks the intended economic design and enables **penalty bypass for low-value stakes**.

The issue originates from the penalty calculation logic used during unstaking:

```
...  
uint256 penalty = (totalStakeAmount * 20) / 100;  
uint256 totalDeductions = penalty + totalClaimedRewards;  
uint256 returnAmount = totalStakeAmount > totalDeductions ? totalStakeAmount  
- totalDeductions : 0;  
...
```

Solidity performs integer arithmetic with truncation, meaning any fractional result is rounded down. As a result:

```
(totalStakeAmount * 20) / 100
```

evaluates to 0.

This causes `penalty == 0`, regardless of the intended 20% fee.

While users cannot unstake multiple times and therefore cannot amplify this issue through repetition, the bug still has notable implications:

- **Incorrect Economic Enforcement:**

The unstaking penalty is not consistently applied across all stake sizes.

- **User Expectation Mismatch:**

Users staking small amounts experience different economic behavior than intended.

Assets:

- StakingAndRewards.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Medium
Severity:	Info

Recommendations

Remediation: It is recommended to:

- enforce a minimum stake amount in the `stake()` function.
- clearly document it If zero-penalty unstaking for small values is intentional.

Resolution: **Fixed** in commit `f9604f9`. The codebase introduced a `MIN_STAKE_AMOUNT` and enforced it in `stake()`, preventing trivial low-value stakes where integer truncation would reduce the unstake penalty to zero and thereby preserving consistent penalty enforcement.

F-2026-14504 - Public Functions Not Used Internally Can Lead To Additional Gas Costs - Info

Description:

In Solidity, function visibility is an important aspect that determines how and where a function can be called from. Two commonly used visibilities are `public` and `external`. A `public` function can be called both from other functions inside the same contract and from outside transactions, while an `external` function can only be called from outside the contract.

Affected functions:

- `StakingAndRewards::getPendingRewards()`
- `StakingAndRewards::getActiveStakes()`
- `Subscription::getReferrers()`
- `Subscription::getLevelInfo()`
- `Subscription::getLevelPaymentInfo()`

A potential pitfall in smart contract development is the misuse of the `public` keyword for functions that are only meant to be accessed externally. When a function is not used internally within a contract and is only intended for external calls, it should be labeled as `external` rather than `public`.

Assets:

- StakingAndRewards.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]
- Subscription.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Declare functions that are not called internally within the contract and are intended for external access as `external` rather than `public`.

Resolution:

Fixed in commit `f9604f9`. The codebase updated the affected view functions to use `external` visibility where they are not called internally, improving gas efficiency for external calls and aligning the interfaces without changing functional behavior.

[F-2026-14508](#) - Missing Zero-Value Validation for directReward Can Trigger Redundant Transfers - Info

Description: In the `_processDirectReward()` function, the `directReward` amount is calculated as a percentage of the staked amount.

```
function _processDirectReward(address user, uint256 amount) internal {
    /* Skip direct rewards if protection system is active */
    if (isProtectionSystemActive()) {
        return; /* Skip direct rewards processing */
    }

    uint256 userId = affiliate.userIds(user);
    if (userId == 0) return;

    uint256[12] memory referrers = affiliate.getReferrers(userId);
    uint256 referrerId = referrers[0];

    if (referrerId != 0 && referrerId != 1) {
        address referrer = affiliate.getUserAddress(referrerId);
        if (referrer != address(0) && affiliate.isMainActive(referrerId))
        {
            uint256 directReward = (amount * DIRECT_REWARD_PERCENT) / PERCENT_DENOMINATOR;

            IERC20(aceToken).transfer(referrer, directReward);
            totalDirectRewards[referrer] += directReward;
            totalDirectRewardsReceived[referrer] += directReward;
            totalWithdrawn += directReward;
            emit DirectRewardPaid(referrer, directReward);
        }
    }
}
```

However, there is no explicit validation ensuring that the calculated `directReward` is greater than zero before attempting to transfer tokens to the referrer. If amount is sufficiently small, integer division may result in `directReward == 0`, yet the contract will still execute a token transfer and emit a `DirectRewardPaid` event.

Impact:

- Causes redundant zero-value token transfers, increasing gas consumption unnecessarily.
- Emits misleading `DirectRewardPaid` events with zero amounts, which may confuse off-chain indexers, analytics tools, or accounting

systems.

Assets:

- StakingAndRewards.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

It is recommended to:

- move the direct reward calculation logic outside of the loop and execute the loop only if the reward is greater than zero:

```
function _processDirectReward(address user, uint256 amount) internal {
    /* Skip direct rewards if protection system is active */
    if (isProtectionSystemActive()) {
        return; /* Skip direct rewards processing */
    }

    uint256 userId = affiliate.userIds(user);
    if (userId == 0) return;

    uint256 directReward = (amount * DIRECT_REWARD_PERCENT) / PERCENT_DENOMINATOR;

    uint256[12] memory referrers = affiliate.getReferrers(userId);
    uint256 referrerId = referrers[0];

    if (referrerId != 0 && referrerId != 1 && directReward > 0) {
        address referrer = affiliate.getUserAddress(referrerId);
        if (referrer != address(0) && affiliate.isMainActive(referrerId))
        {
            IERC20(aceToken).transfer(referrer, directReward);
            totalDirectRewards[referrer] += directReward;
            totalDirectRewardsReceived[referrer] += directReward;
        }
    }
}
```

```
totalWithdrawn += directReward;
emit DirectRewardPaid(referrer, directReward);
    }
}
}
```

- introduce a minimum staking value to avoid zero direct rewards.

Resolution:

Fixed in commit [f9604f9](#). The codebase now computes `directReward` upfront and exits early when it is zero, preventing redundant zero-value ERC20 transfers and avoiding misleading `DirectRewardPaid` events for very small stake amounts.

[F-2026-14509](#) - Confusing Error Handling When User Has No Active Stakes - Info

Description:

The `claimRewards()` function does not explicitly validate that the caller has at least one active stake before proceeding with reward calculations.

When a user without active stakes calls `claimRewards()`, the function:

- Iterates over both stake slots,
- Computes `userTotalStaked` and `totalClaimed` as zero,
- Reaches the `require(totalClaimed < (userTotalStaked * MAX_REWARD_MULTIPLIER) / PERCENT_DENOMINATOR, "Maximum rewards claimed");`
- Since `0 < 0` leads to false, the function reverts with the message **"Maximum rewards claimed"**.

```
function claimRewards() external onlyActive nonReentrant {
    require(block.timestamp >= lastClaimTime[msg.sender] + HOURS, "Cooldown active");

    /* Cache user data to avoid multiple storage reads */
    address user = msg.sender;
    uint256 userTotalStaked = 0;
    uint256 totalClaimed = 0;

    /* Calculate user totals in gas-efficient loop */
    for (uint i = 0; i < 2; i++) {
        if (activeStakes[user][i].active) {
            userTotalStaked += activeStakes[user][i].amount;
            totalClaimed += activeStakes[user][i].totalClaimed;
        }
    }

    require(totalClaimed < (userTotalStaked * MAX_REWARD_MULTIPLIER) / PERCENT_DENOMINATOR, "Maximum rewards claimed");
    ...
}
```

As a result, the transaction produces no meaningful error, leaving the user without feedback on why the claim failed.

Impact

- Leads to **poor UX**, as users receive no indication that they must have active stakes to claim rewards.
- Causes **silent no-op transactions**, wasting gas without providing actionable feedback.

- Makes off-chain monitoring and debugging harder.

Assets:

- StakingAndRewards.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:**Fixed****Classification****Impact Rate:** 1/5**Likelihood Rate:** 1/5**Exploitability:** Independent**Complexity:** Simple**Severity:** Info**Recommendations****Remediation:**

Add an explicit validation that ensures the user has at least one active stake before proceeding:

```
error NoActiveStakes();

function claimRewards() external onlyActive nonReentrant {
    ...
    require(_hasAnyActiveStakes(msg.sender), NoActiveStakes());
    ...
}
```

Resolution:

Fixed in commit [f9604f9](#). The codebase added an explicit `_hasAnyActiveStakes()` validation at the start of `claimRewards()`, reverting early with a clear “No active stakes to claim rewards” message and preventing the misleading “Maximum rewards claimed” error for users without active stakes.

[F-2026-14511](#) - Redundant Zero-Validation of Pre-Initialized lastClaimTime Wastes Gas Across Reward Functions - Info

Description:

The `StakingAndRewards` contract contains an unreachable conditional check that tests whether `startTime == 0` in different reward calculation functions. This condition is **impossible to satisfy** because `lastClaimTime` (which is assigned to `startTime`) is always initialized to `block.timestamp` during stake creation and is only ever updated to `block.timestamp` values thereafter. This represents dead code that wastes gas on every reward calculation and suggests incomplete or misunderstood contract logic. The problematic pattern appears in three functions:

- `StakingAndRewards::_calculateRewardAmount()` (line 553)
- `StakingAndRewards::_calculateSingleReward()` (line 672)
- `StakingAndRewards::_calculateViewRewardAmount()` (line 833)

For instance:

```
...
if (startTime == 0) {
    startTime = stakeEntry.startTime;
}
...
```

the `_processTeamRewards()` function contains the following code:

```
...
if (reward.lastClaimTime == 0) {
    reward.lastClaimTime = currentTime;
}
...
```

Impact:

1. **Gas Wastage:** Each of these three functions performs an unnecessary comparison operation (`startTime == 0`) on every execution. For `_calculateSingleReward()` and `_calculateRewardAmount()`, this happens on **every reward claim** for each active stake (up to 2 per user). For a user with 2 stakes claiming rewards, this means 2 unnecessary SLOAD operations and comparisons per claim.
2. **Code Maintainability Issues:** The presence of this dead code with misleading comments ("For first claim, use stake start time") suggests a misunderstanding of the contract's state management. This makes the codebase harder to audit and maintain, as future developers may

waste time trying to understand when this condition could be triggered.

3. **Logic Confusion:** The comment suggests the developer intended different behavior for the "first claim" vs subsequent claims, but this distinction is not actually implemented. The current implementation **always** calculates rewards from `lastClaimTime`, even on the first claim (where `lastClaimTime` equals `startTime` anyway due to line 249).

Assets:

- StakingAndRewards.sol [<https://github.com/AMGAceToken/Ace-SmartContracts>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: It is recommended to remove the unreachable conditional checks from all the aforementioned functions and simplify the logic or redesign it with the usage of the last payment timestamp.

Resolution: **Fixed** in commit `825eaaf`. The unreachable branches were removed from the codebase.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/AMGAceToken/Ace-SmartContracts
Commit	ac6261b18edc783a17770764280229bb099fdb61
Remediation Commit	825eaafb269566ff3f5e64d1d5cb1f336ee3bfe1
Whitepaper	N/A
Requirements	README.md
Technical Requirements	README.md, Natspec

Asset	Type
AceCoin.sol [https://github.com/AMGAceToken/Ace-SmartContracts]	Smart Contract
MemoryManager.sol [https://github.com/AMGAceToken/Ace-SmartContracts]	Smart Contract
StakingAndRewards.sol [https://github.com/AMGAceToken/Ace-SmartContracts]	Smart Contract
Subscription.sol [https://github.com/AMGAceToken/Ace-SmartContracts]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.